

Technical Report

Project: PHIL Medium Exchange
Company: Applied Nanotechnology
Research Group

Place, date: Enschede, 25th March 2026

Drawn up by: Catalin Chiprian

Version: 1

Foreword

This report was written as part of my internship at the Applied Nanotechnology Research Group (ANT) at Saxion University of Applied Sciences.

During this project, I worked on improving the PHIL pipetting robot, focusing on firmware, software, and system integration. The goal was to make the system more accurate, reliable, and usable for automated medium exchange.

I would like to thank my supervisor and the team at ANT for their guidance, support, and feedback throughout this project. Their help was very valuable during both development and testing.

Finally, I would also like to thank everyone who contributed to testing the system and provided useful feedback on the software.

Summary

This project focuses on improving the PHIL pipetting robot used in the Applied Nanotechnology Research Group (ANT). The goal of the project was to make the system more accurate and reliable for automated medium exchange.

At the start of the project the system had several limitations: the positioning accuracy was not sufficient, medium exchange was not implemented, and the software interface was limited. Additionally, the communication between the software and firmware was not well structured. To address these issues, improvements were made in three areas: hardware, firmware, and software.

On the hardware side, wiring was improved, mechanical issues were fixed, and pumps and reservoirs were integrated into the system. A real-time clock (RTC) module was also added to support scheduled actions.

In the firmware, the origin calibration procedure was redesigned to make it more reliable and repeatable. Position persistence was added using EEPROM, allowing the system to remember its position after restart. A dynamic well positioning method was implemented using calibration points and polynomial mapping. In addition, support for action-based operation and scheduling was added.

The software was redesigned using a structured architecture based on MVVM. The user interface was improved to make it clearer and easier to use. Also, a services layer was introduced to separate logic from the interface, and new features such as calibration, scheduling, and settings management were implemented.

Ultimately, the system was tested for each area. Hardware tests confirmed reliable movement and pump operation. Firmware tests verified correct behaviour of homing, origin calibration, memory, and communication. Software testing showed that all features worked as expected and were usable by end users.

The results show that the system now achieves reliable positioning and supports automated medium exchange. Although several hardware improvements are still recommended, the system is stable and ready for further use and development.

Contents

Foreword.....	2
Summary.....	3
Abbreviations	7
List of figures and/or tables	8
1. Introduction.....	10
1.1 Background of the Assignment	10
1.2 Problem Definition and Objectives.....	11
1.2.1 Current problems	11
1.2.2 Main objectives of the assignment.....	12
1.3 Scope of Work.....	12
1.3.1 Firmware (Arduino).....	12
1.3.2 GUI (.NET/Avalonia)	12
1.3.3 Hardware & System	12
1.4 Approach & Methodology.....	13
1.5 Outline of the Report	13
2. Analysis / Specifications	15
2.1 System Overview	15
2.2 Hardware	16
2.3 Firmware	18
2.4 Software	18
3. Functional Design	21
3.1 Position Control Function	21
3.2 Calibration Function.....	21
3.3 Fluid Handling Function	21
3.4 Scheduling Function.....	21
3.5 Communication Function.....	21
3.6 State Management Function	21
3.7 User Interaction Function.....	22
4. Technical Design.....	23
4.1 Hardware	23
4.2 Firmware	24
4.3 Software	28

5.	Realisation.....	32
5.1	Hardware	32
5.1.1	Left Switch Malfunction	32
5.1.2	Wiring Improvements	32
5.1.3	Cable Management.....	33
5.1.4	Motor Wiring Consistency	33
5.1.5	Arm Column Fixation.....	34
5.1.6	Z-Axis Driver Behaviour.....	35
5.1.7	Pumps and Reservoirs Integration	36
5.1.8	Real-Time Clock Integration	37
5.2	Firmware	38
5.2.1	Homing and Origin Calibration.....	38
5.2.2	Position Persistence (EEPROM)	40
5.2.3	Movement Reliability	40
5.2.4	Dynamic Well Positioning.....	40
5.2.5	Action-Based Operation.....	42
5.2.6	Communication Interface	43
5.2.7	Auxiliary Positions.....	44
5.2.8	Firmware Refactoring	44
5.3	Software	44
5.3.1	Front-End Redesign.....	44
5.3.2	Architecture and MVVM Improvements	44
5.3.3	Command Handling and Toolkit Integration	45
5.3.4	Services Layer	45
5.3.5	Settings Module	46
5.3.6	Views and User Interaction.....	47
5.3.7	Startup and Connection Flow.....	51
5.3.8	Documentation and User Support	51
6.	Testing.....	53
6.1	Hardware	53
6.1.1	RTC Module Test.....	53
6.1.2	Pump Control Test	53
6.1.3	Volume Calibration (Steps to μL Conversion)	53

6.1.4	Mechanical movement reliability	55
6.2	Firmware	55
6.2.1	Homing and Origin Calibration.....	55
6.2.2	Position Persistence (EEPROM)	55
6.2.3	Dynamic Well Positioning.....	55
6.2.4	Pump Control Logic	56
6.2.5	Action-Based Operation.....	56
6.2.6	Communication Interface	56
6.2.7	Firmware Refactoring	56
6.3	Software	56
6.3.1	Command Handling and Toolkit Integration	56
6.3.2	Services Layer	57
6.3.3	Settings Module	57
6.3.4	Views and User Interaction.....	57
7.	Conclusions and Recommendations	58
7.1	Conclusion	58
7.2	Recommendations	58
7.2.1	Hardware Improvements	58
7.2.2	Firmware Improvements	59
7.2.3	Software Improvements	59
7.2.4	PCB and Electrical Improvements	59
8.	Use of Tools	60
9.	References.....	61
10.	Appendix	62

Abbreviations

UX: User Experience

UI: User Interface

ANT: Applied Nanotechnology

GUI: Graphical User Interface

RTC: Real Time Clock

MVVM: Model-View-ViewModel

List of figures and/or tables

Figure 1.1 PHIL	10
Figure 1.2 Standard 96 Well Plate Schematic Dimensions	11
Figure 1.3 Scrum Board	13
Figure 2.1 PHIL Top Body	15
Figure 2.2 PHIL Bottom Body	15
Figure 2.3 Custom PCB mounted on the Arduino	16
Figure 2.4 Solder pins to jumper (source: [1])	17
Figure 2.5 Individually insulated jumper wires with headers (source: [1]).....	17
Figure 2.6 Grouped insulated jumper wires with headers (source: [1]).....	17
Figure 2.7 MainWindow with Serial Monitor (source: [1])	19
Figure 2.8 BasicControlsView (source: [1])	19
Figure 2.9 WellsView (source: [1])	20
Figure 2.10 GUI Diagram (source: [1])	20
Figure 4.1 Electrical Low Level Diagram (source: Adapted from [1])	23
Figure 4.2 High Level Firmware Diagram	25
Figure 4.3 Low Level Arduino Diagram.....	27
Figure 4.4 Low Level Modules Relationship Diagram.....	28
Figure 4.5 High-level GUI workflow of the software	29
Figure 4.6 Backend Services and their relationships.....	30
Figure 4.7 Software Low Level Diagram.....	31
Figure 5.1 L-Switch Electrical Connection viewed from the PCB Editor View.....	32
Figure 5.2 L-Switch Electrical Connection viewed in real.....	32
Figure 5.3 Improved wiring with crimped connectors and protective housing.....	33
Figure 5.4 Organized wiring with zip ties and grouped cables	33
Figure 5.5 Inconsistent motor wiring order causing incorrect motor direction.....	34
Figure 5.6 Standardised motor wiring order after correction.....	34
Figure 5.7 Arm column on motor shaft.....	35
Figure 5.8 Arm column with missing screw	35
Figure 5.9 Pump Motors connected to Pump Drivers	36
Figure 5.10 Integrated Pumps and Reservoirs.....	37
Figure 5.11 Integrated RTC module.....	38
Figure 5.12 Homing and origin calibration procedure based on limit switches.....	39
Figure 5.13 Polynomial mapping based on calibration points.	41
Figure 5.14 Actions Process Diagram	43
Figure 5.15 Settings Module, PlateView.....	46
Figure 5.16 Settings Module, ControlsView.....	46
Figure 5.17 Settings Module, DebugView	47
Figure 5.18 WellsView	47
Figure 5.19 CalibrationView	48
Figure 5.20 Pump Control Window	49
Figure 5.21 MediumScheduleView	49
Figure 5.22 ActionWindow for Organ-on-Chip	50
Figure 5.23 ActionWindow for 96-Well	50

Figure 5.24 MainWindow.....	51
Figure 5.25 PortsWindow.....	51
Figure 6.1 Time Serial Monitor.....	53
Figure 6.2 Small plastic container.....	54
Figure 6.3 Tared scale.....	54
Table 6.4 Measured results	55
Figure 10.1 Action execution recording (1-minute interval)	62
Figure 10.2 Action execution recording (1-hour interval)	62
Figure 10.3 Action execution recording (30-minute interval)	63
Figure 10.4 Action execution recording (2-day interval).....	63

1. Introduction

1.1 Background of the Assignment

The Applied Nanotechnology Research Group (ANT) is part of Saxion University of Applied Sciences and focuses on applied research in nanotechnology. The main research areas within ANT include molecular sensing, organ-on-chip technology, and circularity. These research activities are carried out in close collaboration with students, researchers, and external partners.

This project belongs to the organ-on-chip research line. The goal is the development of a high-throughput heart-on-a-chip platform to study human heart tissue responses to medication and disease in a controlled environment. By using human cell-based models, the project aims to reduce the need for animal testing and provides a more ethical and reliable alternative for drug research.

A key part of the heart-on-a-chip platform is the automated exchange of culture medium. This process must be accurate, repeatable, and compatible with incubator conditions, more specifically around 37°C and approximately 80% humidity. For this purpose, a small-scale pipetting robot called PHIL (Pipetting Helper Imaging Lid) is used (see Figure 1.1). PHIL is designed to move above a well plate, position a pipette accurately, and perform liquid handling tasks.

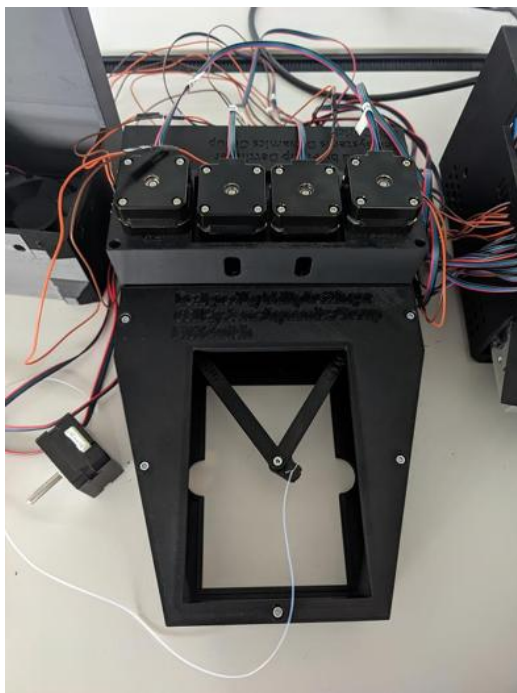


Figure 1.1 PHIL

PHIL works using three basic processes: origin calibration, calibration and medium exchange. Origin calibration is the step where the robot moves to a known starting position using limit switches. This ensures that all movements are based on a correct reference point. Further, calibration is the process of linking the physical positions of the wells to the motor positions, so the pipette can move accurately to each well.

A well plate is a container used in laboratories, consisting of multiple small wells arranged in a grid. A common format is the 96-well plate, which has 8 rows and 12 columns, with a standard center-to-center

spacing of 9 mm between wells (see Figure 1.2). The plate itself is approximately 127.6 mm long and 85.5 mm wide. Each well holds liquid or biological samples.

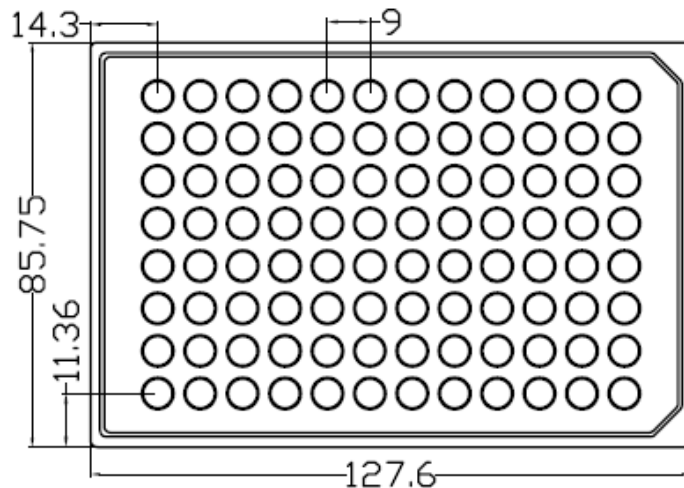


Figure 1.2 Standard 96 Well Plate Schematic Dimensions

Pipetting is the process of taking in and dispensing liquids. Medium exchange is a specific type of pipetting where old liquid is removed from a well and replaced with fresh liquid to keep cells healthy.

It is worth mentioning that this project is a continuation of an earlier internship project. In the previous project, the main focus was on repairing and redesigning the electronics, firmware, and user interface of the PHIL robot [1]. That project resulted in a new PCB design, a rewritten Arduino firmware, and a new .NET-based GUI. Moreover, basic robot movement and origin calibration were achieved. However, the origin calibration lacked efficiency that resulted in slow operation and inconsistent positioning. Moreover, precise well positioning and automated pipetting were not fully realized within the available timeframe.

1.2 Problem Definition and Objectives

Although PHIL already has a new PCB, updated firmware, and a basic GUI from the previous internship, the system is still not complete. Several essential functions are missing or insufficiently developed for use in the heart-on-a-chip platform. The GUI requires a full front-end and back-end redesign, and the communication between the GUI and the Arduino must be restructured to support reliable two-way data exchange. In addition, the robot must be able to perform accurate (± 0.5 mm) well positioning and automated medium exchange, and the reservoir and waste container must be integrated into the system.

1.2.1 Current problems

- The well-positioning accuracy does not meet the required precision of ± 0.5 mm for reliable automated pipetting.
- Medium-exchange routines are not implemented.
- The reservoir and waste container are not yet integrated.
- The GUI lacks functions for medium exchange, calibration, and improved user feedback.

1.2.2 Main objectives of the assignment

- Achieve a repeatable well-positioning accuracy of ± 0.5 mm or better.
- Implement reliable firmware routines for medium exchange.
- Redesign the GUI front-end with a clearer layout and improved usability.
- Redesign the GUI back-end to support structured two-way communication and better error handling.
- Ensure stable and reliable long-term operation of the system.
- Document all changes for future developers.

The goal is to deliver a stable, accurate, and usable PHIL system that researchers can rely on for automated medium exchange.

1.3 Scope of Work

This internship focuses on completing and improving the PHIL pipetting robot so it can be used in the heart-on-a-chip well plate.

1.3.1 Firmware (Arduino)

- Implement reliable well positioning logic.
- Ensure repeatable origin calibration using limit-switch detection and defined zero positioning.
- Implement pipetting and medium-exchange routines.
- Add structured status reporting for the GUI (e.g., position, well information, calibration values).

1.3.2 GUI (.NET/Avalonia)

Front-end redesign:

- Develop a new, clearer layout.
- Redesign the structure of controls to improve usability.
- Create a new calibration page.
- Create a new medium-exchange page.

Back-end redesign:

- Rework the communication layer to support structured incoming and outgoing messages.
- Implement more stable message handling and error detection.
- Provide consistent feedback and status updates to users.
- Replace old code from the previous version with a cleaner and more maintainable architecture.

1.3.3 Hardware & System

- Verify wiring, limit switches, and overall mechanical operation.
- Integrate the reservoir and waste container into the system.
- Identify and resolve existing hardware-related issues that affect stability.

Only improvements necessary for a functional and stable PHIL system are included. Full mechanical redesigns or biological experiments are outside the project scope.

1.4 Approach & Methodology

The project follows a modified Scrum-based approach suitable for technical work. All tasks are organised using a GitHub Kanban board with four columns: Backlog, In Progress, In Review, and Done (see Figure 1.3). This structure helps to divide the work into small, manageable steps and makes it possible to adjust the planning whenever new technical issues arise during testing.

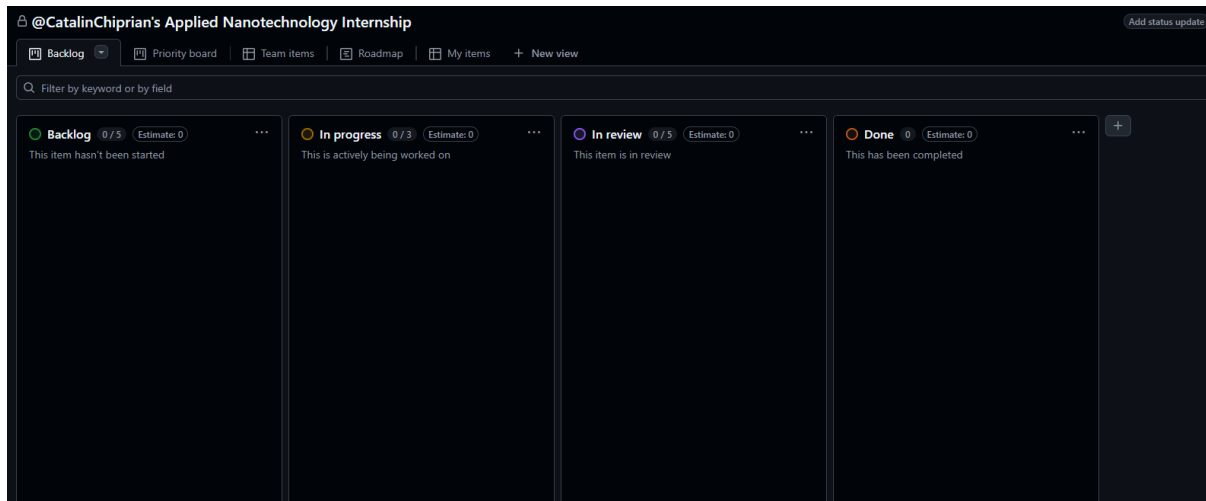


Figure 1.3 Scrum Board

Also, work is carried out in short cycles. In each cycle, small parts of the firmware, GUI, or hardware functionality are improved, tested, and refined based on the results. Weekly meetings with the technical supervisor are used to discuss progress, review completed tasks and plan the next steps.

Since both hardware and software components influence each other, the methodology focuses on iterative development and continuous testing. Firmware updates, GUI changes, and mechanical adjustments are tested immediately so that errors or limitations can be discovered early.

1.5 Outline of the Report

Chapter 1 – Introduction: Provides background information, defines the problem and objectives, outlines the project scope, and explains the methodology.

Chapter 2 – Analysis and Specifications: Describes the existing system, the limitations identified at the start of the internship, and the requirements for further development.

Chapter 3 – Functional Design: Presents the intended system behaviour, including the planned user functions, workflows, and interface structure.

Chapter 4 – Technical Design: Describes the technical architecture of the system, including the structure of the firmware, the GUI, and the communication between components.

Chapter 5 – Realisation: Explains the implementation of the planned improvements, including software development, system adjustments, and integration work.

Chapter 6 – Testing: Summarises the testing methods and results, covering unit tests, integration tests, system tests, and other relevant validation steps.

Chapter 7 – Conclusions and Recommendations: Summarises the project outcomes, evaluates the achieved objectives, and provides recommendations for future work.

2. Analysis / Specifications

2.1 System Overview

The PHIL robot consists of a mechanical system with two stepper motors controlling horizontal movement and two additional motors for Z-axis positioning. Movement in the XY plane is controlled using coordinate-based commands for positioning the pipetting head. Limit switches are used to determine reference positions during the origin calibration process. Figures 2.1 and 2.2 show the main mechanical components of the system, including the motor assembly and the positioning mechanism.



Figure 2.1 PHIL Top Body

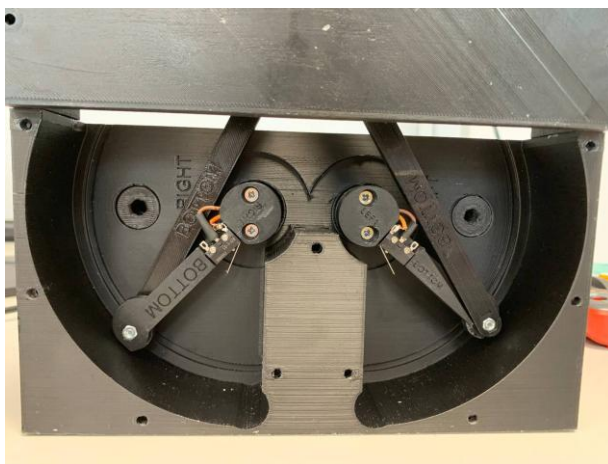


Figure 2.2 PHIL Bottom Body

The system is controlled by an Arduino, which handles motor control, limit switch input, and communication with the GUI. The electronics are implemented on a custom-designed PCB mounted on the Arduino, which acts as an interface between the controller and the hardware. The PCB integrates

motor driver modules and connections for the motors and limit switches. Figure 2.3 shows the custom PCB used in the system.

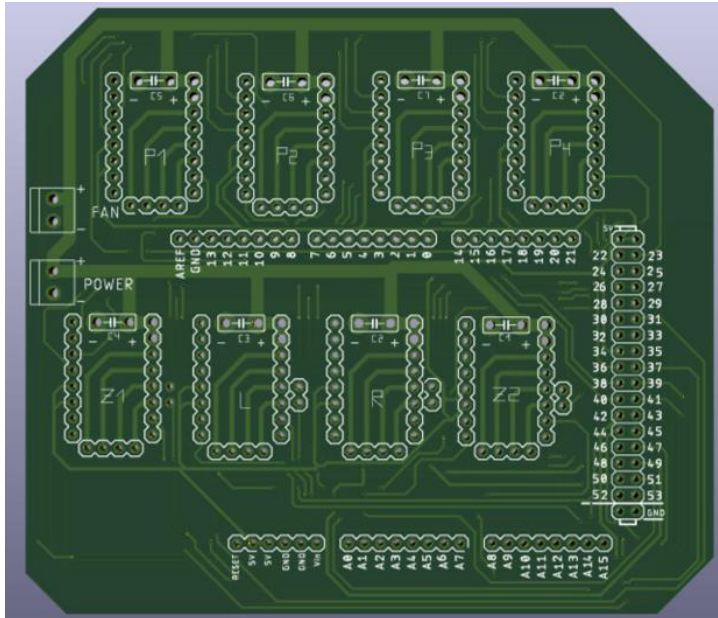


Figure 2.3 Custom PCB mounted on the Arduino

2.2 Hardware

The hardware of the PHIL system had several issues that affected safety, reliability, and usability. One of the main problems was the limit switch setup. The left limit switch could not be used because it caused a short circuit that could shut down the Arduino. For this reason, it was disconnected, and only the right limit switch was used for origin calibration. This made the system usable, but it reduced reliability because there was no backup if the right switch failed.

Since the limit switch alone could not guarantee a correct reference position, the system was homed by moving towards the wall for a fixed amount of time. This resulted in the pipetting head physically contacting the structure, ensuring that a consistent reference position was reached.

The wiring of the system was also not suitable for long-term use. The jumper wires were soldered to header pins (see Figure 2.4) and insulated with tape (see Figure 2.5). While grouping these wires kept the bundle managed, it remains an unreliable long-term solution (see Figure 2.6). This is especially true in an incubator environment, where higher temperature and humidity can degrade the tape insulation over time.



Figure 2.4 Solder pins to jumper (source: [1])

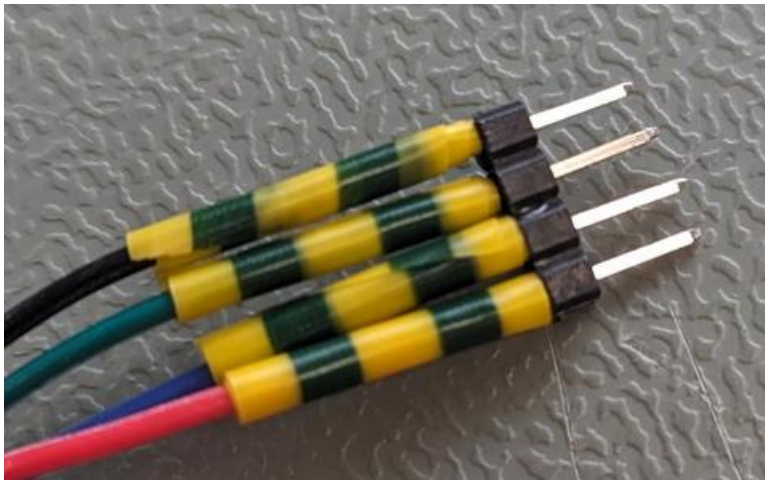


Figure 2.5 Individually insulated jumper wires with headers (source: [1])



Figure 2.6 Grouped insulated jumper wires with headers (source: [1])

Another issue was the mechanical stability of the system. After disassembling and reassembling the robot, the movement directions of the motors changed. The exact cause was not found, which shows that the system is sensitive to mechanical changes. This indicates a lack of mechanical stability, and disassembling the robot can lead to unpredictable behaviour.

The hardware configuration also had problems with the motor drivers. The Z1 and Z2 motor drivers did not work correctly and had to be connected to the pump drivers P1 and P2. In addition, disconnecting one motor could affect the behaviour of the system, which shows that the power and connections were not stable.

The overall hardware setup was not fully integrated. The pumps and reservoirs were not part of the system and were not included in the initial setup. This meant that the system was incomplete and could not be used for automated liquid handling.

2.3 Firmware

The firmware provided basic functionality but had several limitations that affected performance and reliability [1]. The system used a wall-based origin calibration method, where the robot moved until it reached the limit switch and then continued moving for a short time to ensure alignment. This relied on timed contact with a physical boundary.

This origin calibration approach was slow and not reliable. Because it depended on mechanical contact and timing, the final position could vary slightly each time. Even small differences can be a problem for precise pipetting. In addition, this method caused mechanical stress and unnecessary noise.

The system required recalibration before almost every movement to achieve acceptable positioning accuracy. This made the system slow, with movements sometimes taking tens of seconds. Removing calibration steps made the system faster, but also less accurate, which shows that the system depended on these corrections instead of precise control.

Well positioning was implemented using hardcoded coordinates. These positions worked from a software perspective, but they did not adapt to mechanical variation. As a result, the robot could not consistently move to the same position.

The movement behaviour was also influenced by physical conditions such as motor temperature. In some cases, accuracy improved at certain temperatures, but the results were inconsistent. Changing the motor current improved performance slightly but did not solve the main problem.

Basic movement was also not fully reliable. When changing direction, the robot could briefly continue moving in the previous direction before responding correctly. This reduced control accuracy and caused unexpected motion.

2.4 Software

The software consisted of a simple graphical user interface with limited functionality [1]. The GUI included a serial monitor on each page, which allowed reading output from the Arduino. While this was useful for debugging, it was not designed for normal user interaction (see Figure 2.7, Figure 2.8).

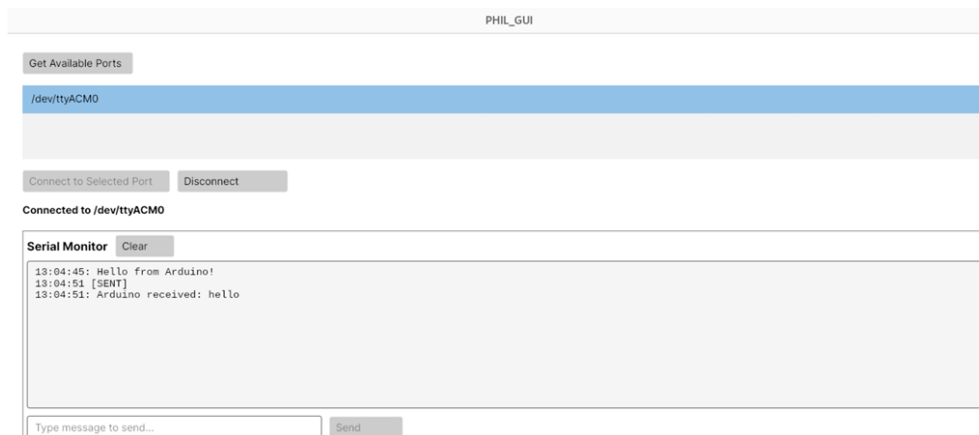


Figure 2.7 MainWindow with Serial Monitor (source: [1])

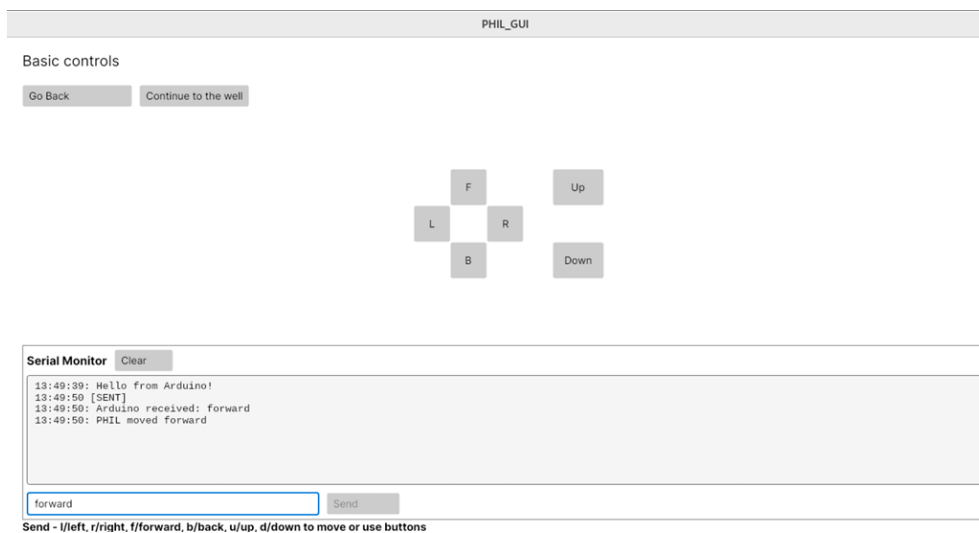


Figure 2.8 BasicControlsView (source: [1])

Another major limitation was the communication between the GUI and the Arduino. The user was able to send commands from the GUI, such as moving the robot in basic directions or selecting a well using hardcoded positions. However, this communication was only one-way. The GUI would send a command, but it did not properly process or react to the response from the Arduino.

In practice, this meant that when the user clicked a button, the command was sent, but there was no structured feedback or confirmation handled by the GUI. The Arduino could send messages back, but the GUI only displayed them in the serial monitor without using them in the program logic. As a result, there was no true two-way communication, and the system lacked reliable control and real-time feedback.

The interface itself mainly consisted of a manually created grid of 96 buttons, each representing a well position (see Figure 2.9). These buttons were directly linked to hardcoded movement commands, which made the system difficult to maintain and not scalable.

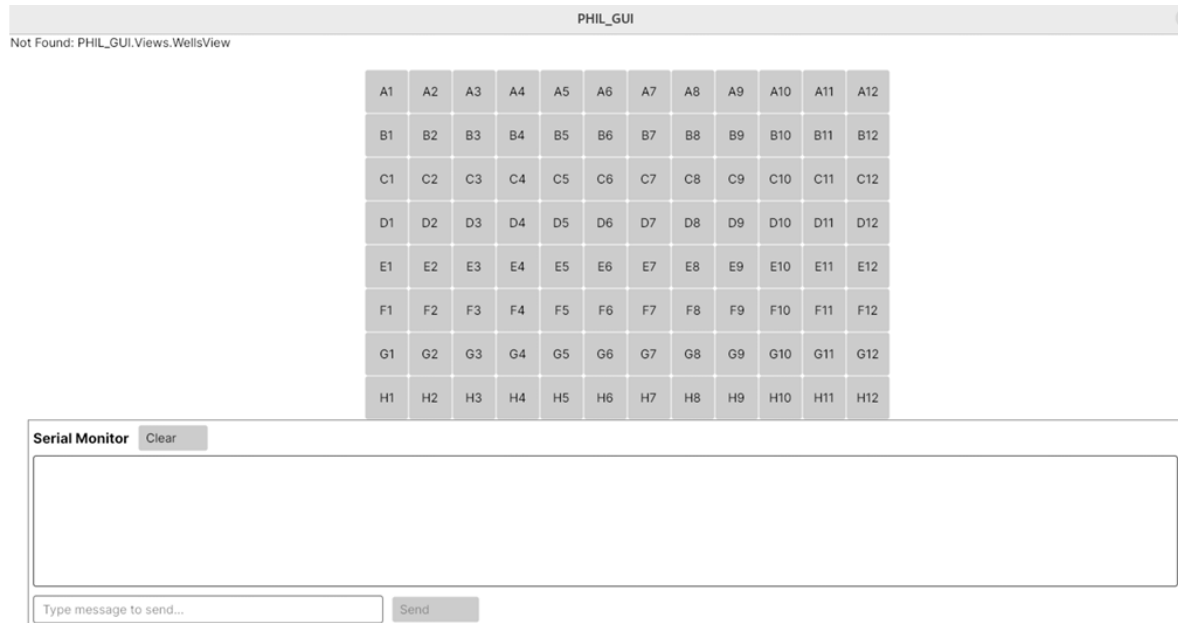


Figure 2.9 WellsView (source: [1])

The structure of the software was also not well designed. As shown in the GUI diagram (see Figure 2.10), the system used separate classes for basic controls, the main window, and well positioning. However, these classes contained repetitive boilerplate code, especially for event handling. This made the code harder to maintain and increased the risk of inconsistencies

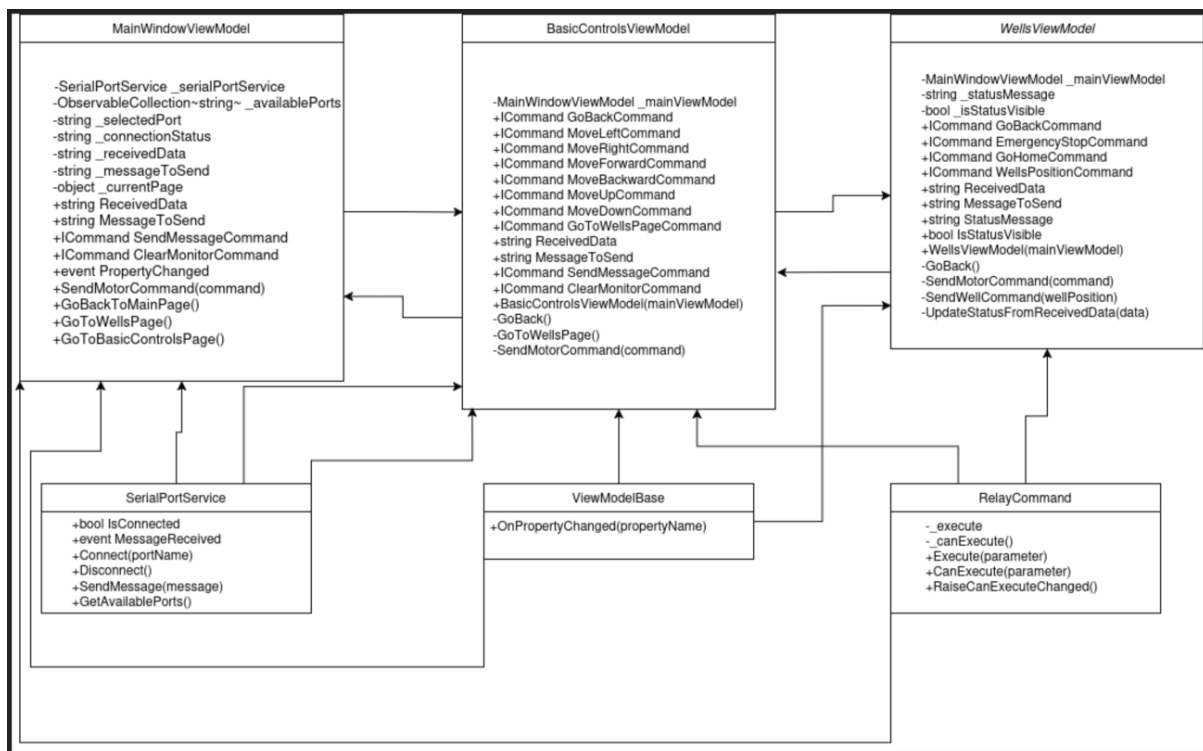


Figure 2.10 GUI Diagram (source: [1])

3. Functional Design

3.1 Position Control Function

The position control function is responsible for determining and controlling the spatial position of the robot within the workspace. This function enables movement between wells, manual positioning, and returning to an origin position. It combines hardware components such as stepper motors and limit switches with firmware logic that generates coordinated motor commands. The software interface allows the user to initiate movement and visualize the current position. The interaction between these components ensures that movement is accurate and repeatable.

3.2 Calibration Function

The calibration function is responsible for improving positioning accuracy by mapping logical well positions to physical coordinates. Calibration data is obtained through user interaction and stored in persistent memory. The firmware uses this data to compute accurate positions during operation, while the software provides user controls to record and manage calibration points. This function compensates for mechanical inaccuracies and ensures that calculated well positions match the real-world layout.

3.3 Fluid Handling Function

The fluid handling function enables liquid operations within the system. It is implemented using pump motors and reservoirs in the hardware, controlled by firmware logic that executes aspirate and dispense operations. The software interface allows the user to trigger these operations manually or as part of automated workflows. This function depends on the position control function, as fluid operations require accurate positioning of the system at the target well.

3.4 Scheduling Function

The scheduling function is responsible for executing operations at specific times. It uses the real-time clock to track system time and determine when actions should be performed. The firmware evaluates scheduled actions and triggers them when conditions are met. The software provides an interface for defining and managing these schedules. This function enables long-term automated operation, including periodic medium exchange.

3.5 Communication Function

The communication function manages the exchange of data between the software and the firmware. Commands are sent from the GUI to control the system, while feedback from the firmware is returned to inform the user about system status and operation results. This function ensures that communication is structured, reliable, and synchronized, allowing the user to monitor and control the system effectively.

3.6 State Management Function

The state management function maintains the internal state of the system, including the current position, selected well, and stored calibration data. It also manages persistent storage using EEPROM to retain important data between sessions. This function ensures continuity in system behaviour and reduces the need for manual reconfiguration after restarts.

3.7 User Interaction Function

The user interaction function provides the interface through which the user controls the system. It includes manual movement controls, pump controls, and scheduling interfaces. The GUI abstracts the complexity of the underlying system and allows users to operate it without requiring detailed technical knowledge. This function ensures ease of use and accessibility.

4. Technical Design

4.1 Hardware

The hardware of the PHIL system is designed as a complete platform that allows the robot to move, position itself accurately, and handle liquids automatically. The system combines motors, pumps, motor drivers, and control electronics such as the Arduino and RTC module that all work together under the control of the Arduino, as shown in Figure 4.1.

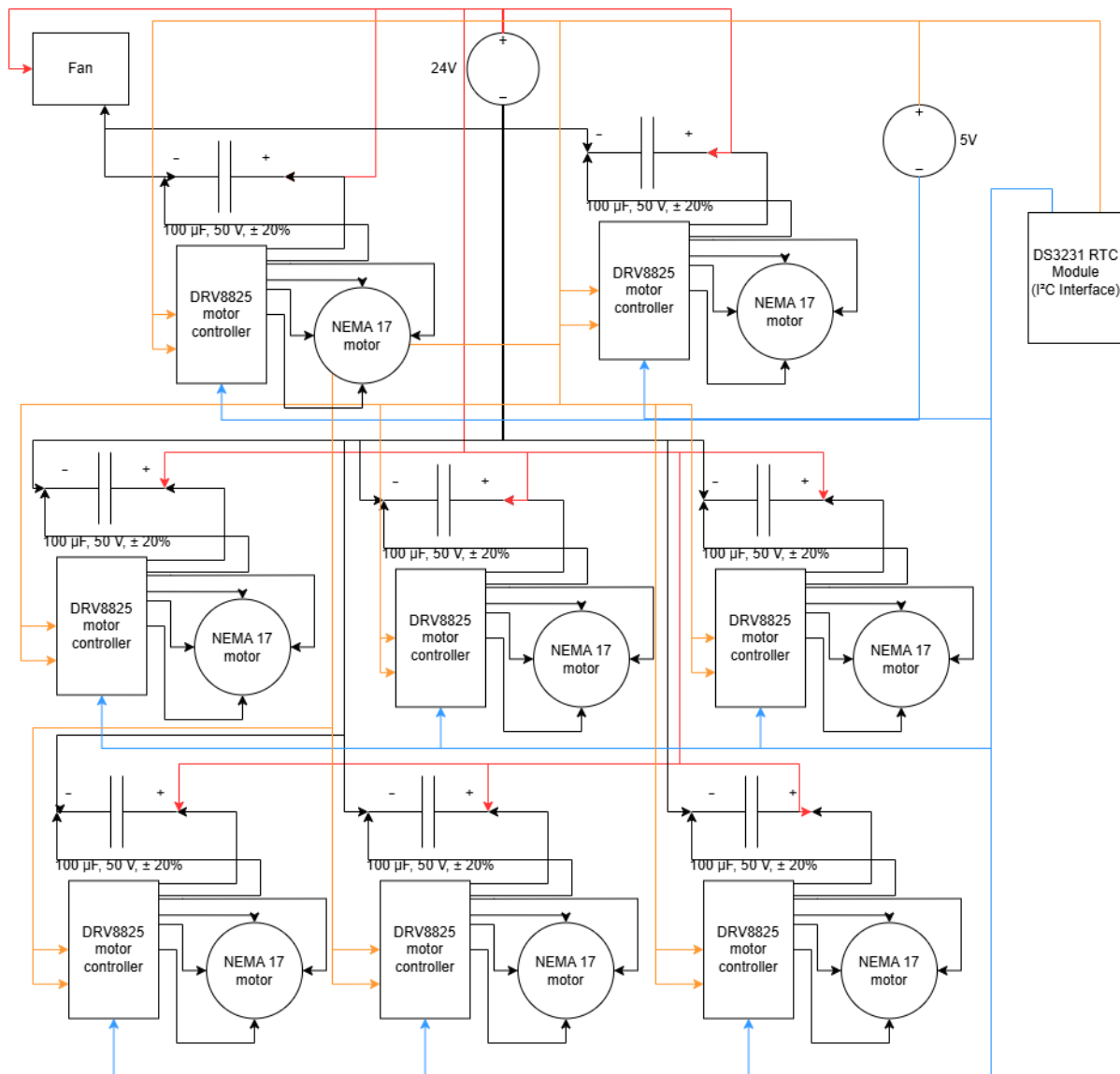


Figure 4.1 Electrical Low Level Diagram (source: Adapted from [1])

Figure 4.1 shows the low-level electrical architecture, where multiple NEMA 17 stepper motors are driven by DRV8825 motor drivers, all connected to a shared power and control system.

The movement of the robot is controlled by several stepper motors. Two motors are responsible for horizontal movement, which allows the robot to move across the well plate. Two additional motors

control the vertical movement of the pipette. These motors are driven by DRV8825 motor driver modules, which receive control signals from the Arduino. As shown in Figure 4.1, each motor is connected to a dedicated driver, allowing precise control over motion and positioning.

To make sure that all movements start from a known position, the system uses limit switches. These switches detect when the robot reaches its origin position. When the system starts, the robot moves towards the switches until they are activated. This creates a reference point that is used for all further movements. The limit switches also provide a form of safety by preventing the robot from moving beyond its physical limits.

The liquid handling system is based on pump motors that are connected to the same type of DRV8825 motor drivers as the movement system. This shared architecture simplifies control and allows the Arduino to operate both positioning and fluid handling using a consistent interface. These pumps are used to aspirate and dispense liquid. They are connected to tubing and reservoirs, which together allow the system to transfer liquid between different locations. This makes it possible to perform automated medium exchange without manual interaction.

The central control of the system is handled by an Arduino microcontroller. The Arduino controls all motors and pumps, reads input from the limit switches, and communicates with the graphical user interface. As illustrated in Figure 4.1, all hardware components are connected to the Arduino through input and output pins, which makes it possible to control the entire system from one central unit.

In addition to movement and liquid handling, the system includes a RTC DS3231 module, as shown in Figure 4.1. This module communicates with the Arduino via an I²C interface, allowing the system to keep track of time independently of the microcontroller. The RTC enables precise scheduling of Actions, which is essential for automated processes such as periodic medium exchange over extended periods.

All components are connected through wiring and powered by a shared power system. The hardware is designed so that all parts operate together as a single integrated system.

4.2 Firmware

The firmware of the PHIL system follows a defined processing flow for handling commands and system behaviour. This flow describes how incoming data from the GUI is received, interpreted, and executed by the system, after which structured feedback is returned. The processing flow of the firmware is shown in Figure 4.2.

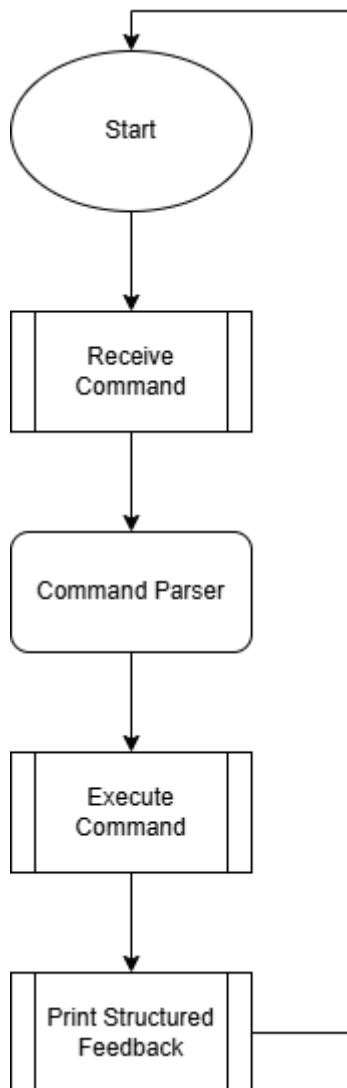


Figure 4.2 High Level Firmware Diagram

In addition to this execution flow, the firmware is designed using a modular structure. Instead of placing all functionality in a single file, the firmware is divided into multiple modules, each with a specific responsibility.

The firmware is organised into two main folders, an inc folder and a src folder. The inc folder contains header files, which define the interfaces of the modules. These header files describe what functions and data are available to other parts of the system. The src folder contains the corresponding source files, where the actual implementation of these functions is written.

Each module represents a specific part of the system. The movement module is responsible for controlling the motors and handling all positioning-related tasks. It includes functions for manual movement, homing, and moving to specific positions or wells. The pumps module handles all liquid-related operations such as aspirating and dispensing. This separation ensures that movement logic and liquid handling logic are independent from each other.

The commands module is responsible for communication with the graphical user interface. It reads incoming serial data, interprets commands, and calls the correct functions from other modules. This

creates a clear separation between input handling and system behaviour. The commands module does not directly control hardware but instead acts as a bridge between the GUI and the rest of the firmware.

The Actions module manages scheduled operations. It defines structures for actions and executes them at the correct time. This module works together with the real-time clock and allows the system to perform automated medium exchange without user interaction.

Calibration and positioning calculations are handled mainly in the calibration module. This module contains the logic that improves positioning accuracy based on stored calibration data and mathematical model that maps positions to motor angles. It also includes the origin calibration functionality.

The well utilities module supports this process by providing helper functions related to well positions and coordinate conversions. These functions handle tasks such as converting between well indices and row-column formats, translating well positions to coordinates, and converting between steps and angles. Unlike the other modules, it does not contain any logic itself but provides basic functions that are used by multiple parts of the firmware.

All persistent data is managed in the EEPROM utilities module. This module is responsible for saving and loading system data such as positions, calibration values, and scheduled Actions. By isolating memory access in a single module, the firmware avoids duplicated code and reduces the risk of errors when working with stored data.

The hardware module provides a low-level interface to the physical components. This includes definitions of pins, motor instances, and functions for enabling or disabling motors. Other modules interact with the hardware through this layer instead of directly accessing pins.

The main Arduino file acts as the entry point of the system. It contains the setup and loop functions, which initialise the system and execute the main program cycle. The loop continuously processes commands, executes scheduled Actions, and updates the system state. Because most functionality is moved into modules, the main file remains small and focused on overall control, as shown in Figure 4.3.

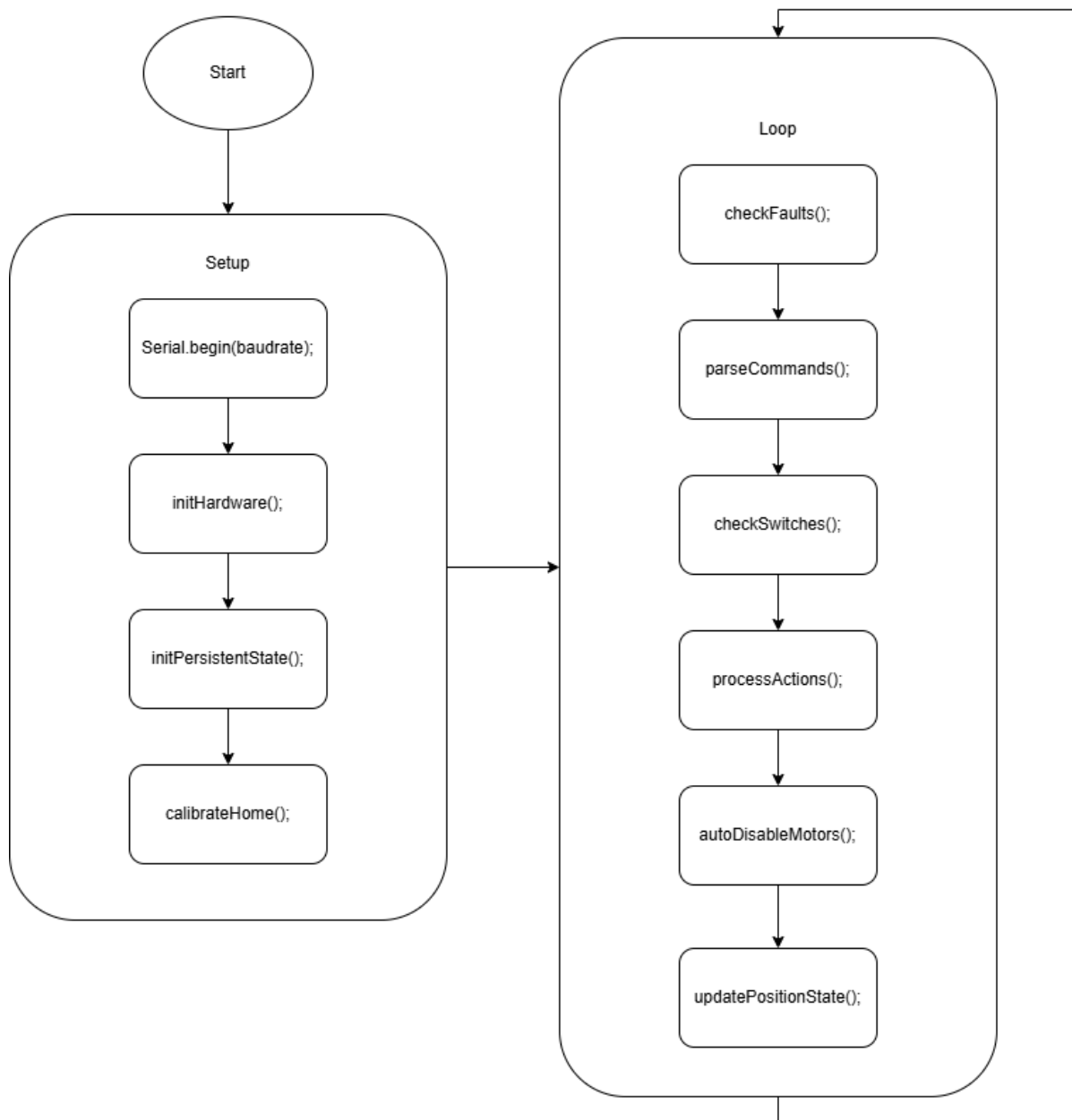


Figure 4.3 Low Level Arduino Diagram

The communication between modules follows a clear structure. Higher-level modules such as commands and actions call functions from lower-level modules such as movement and pumps. Lower-level modules do not depend on higher-level modules, which prevents circular dependencies and keeps the architecture stable, as shown in Figure 4.4.

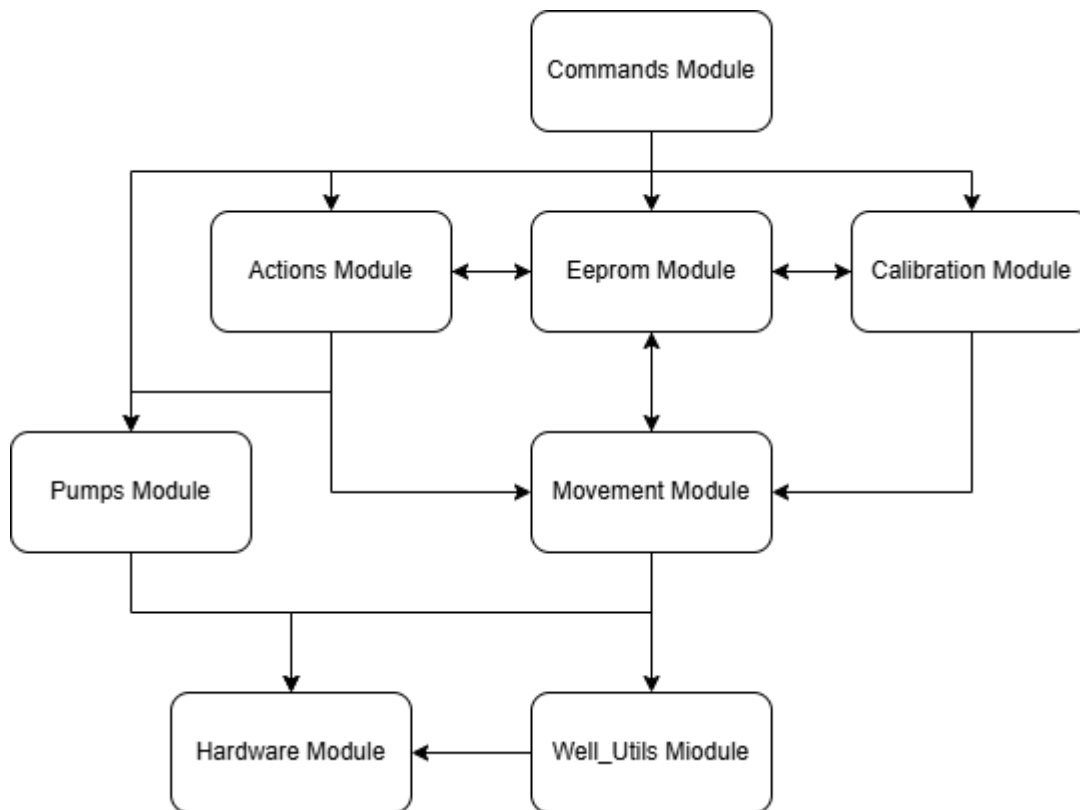


Figure 4.4 Low Level Modules Relationship Diagram

This modular design ensures that each part of the firmware has a clear purpose and a defined interface. As a result, the system is easier to understand, test, and extend.

4.3 Software

The software of the PHIL system consists of GUI that is used to control the robot and monitor its behaviour. The software is developed using .NET Avalonia and follows a structured design that separates user interaction, application logic, and communication with the firmware.

The main purpose of the software is to provide a clear and user-friendly way to operate the system. Instead of interacting directly with the Arduino through raw serial commands, the user can control the system through buttons, views, and visual feedback. This improves usability and reduces the chance of user errors. The structure and navigation flow of the GUI are shown in Figure 4.5.

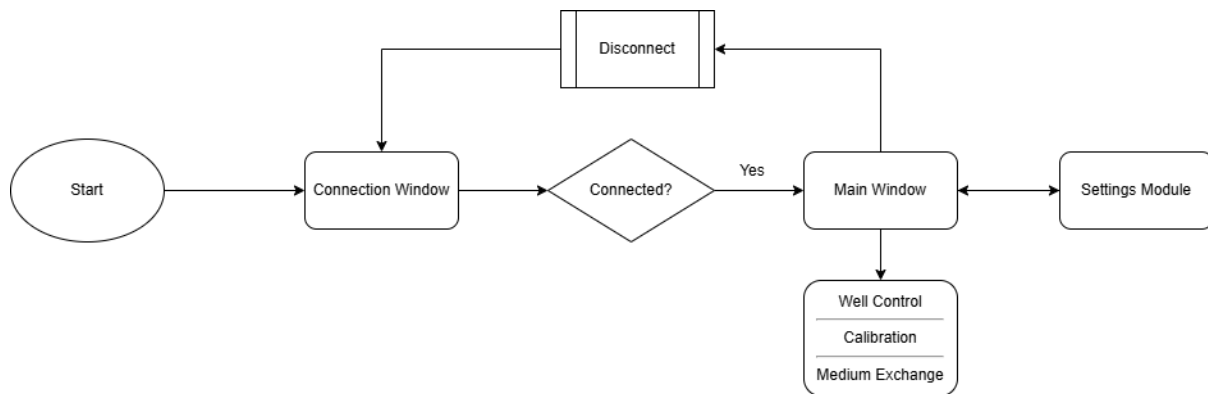


Figure 4.5 High-level GUI workflow of the software

The application is built around a view-based structure. Each part of the system is represented by a separate view, such as well selection, calibration (which also includes basic controls), and medium exchange. These views are responsible for presenting information to the user and allowing the user to trigger actions. By separating the interface into different views, the system remains organised and easier to understand.

Behind the visual interface, the software uses a structured backend that handles application logic and communication. When a user interacts with the interface, such as pressing a button or selecting a well, the software translates this interaction into a command. This command is then sent to the Arduino through a serial connection.

The backend of the application is further structured using services. These services handle specific responsibilities such as communication, application settings, and system state. The system state represents the current condition of the robot, including position, calibration data, and scheduled Actions. For example, a serial communication service manages the connection to the Arduino, while a protocol service translates commands and processes incoming messages. A settings service is used to store and load configuration data locally. A media service is responsible for recording and uses the IRecordContext interface to access the required data without depending on specific implementations.

By separating this logic into services, the system avoids placing complex functionality directly in the user interface layer and remains easier to maintain and extend.

The relationships between these services and their responsibilities are illustrated in the class diagram shown in Figure 4.6. The RobotProtocolService acts as the central component, coordinating communication and updating the system state. It uses the SerialPortService to send and receive data from the Arduino and updates the RobotState based on incoming messages. The AppSettingsService manages application configuration through the AppSettings class, which implements the IRecordContext interface used by the MediaService.

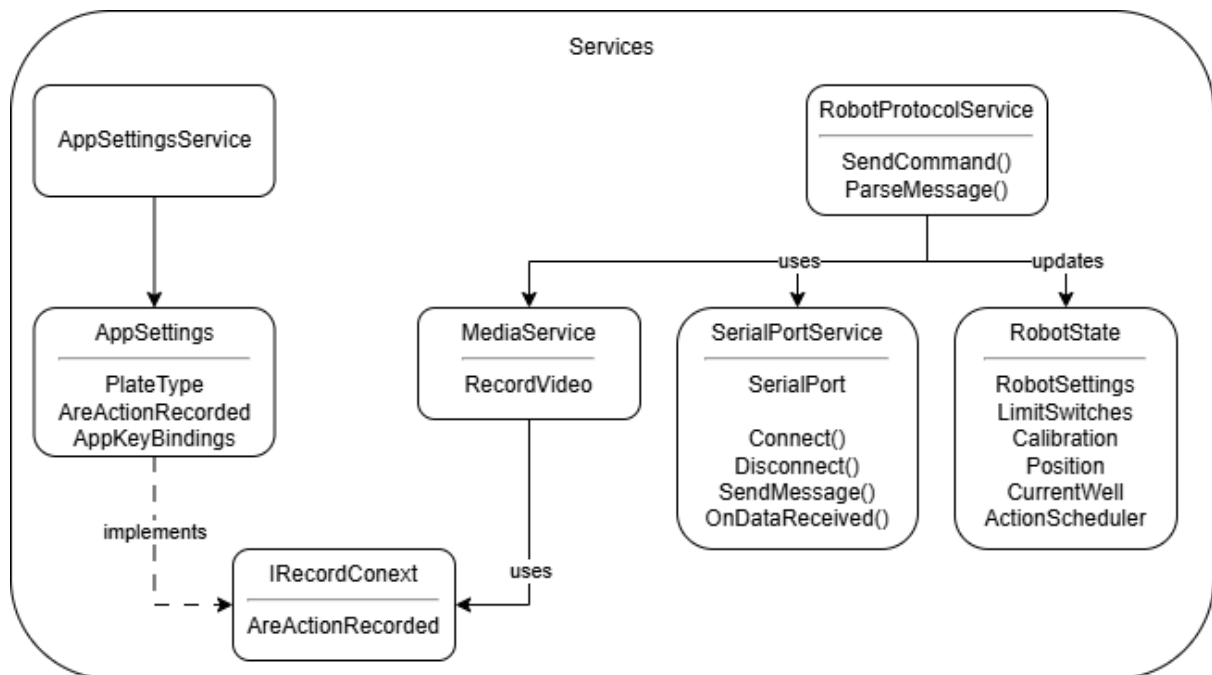


Figure 4.6 Backend Services and their relationships

The application uses view models to connect the user interface with the backend logic. These view models handle user interactions and use the services to perform actions, ensuring a clear separation between the interface and the underlying logic. The overall structure of the software architecture is shown in Figure 4.7.

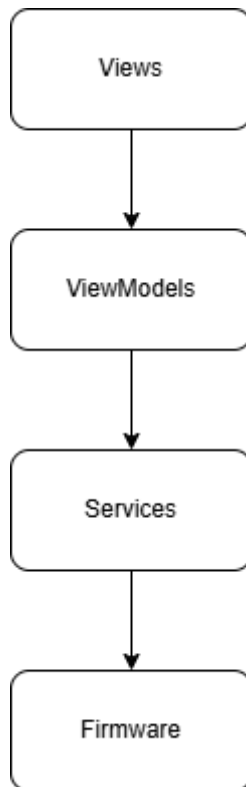


Figure 4.7 Software Low Level Diagram

Commands are sent from the GUI to the Arduino, where they are executed by the firmware. Depending on the type of command, the firmware may provide feedback in the form of data updates, such as position changes or selected well, or debug output messages. Not all commands produce explicit structured responses, as some operations only result in internal state changes or simple printed feedback.

The system also includes a dedicated connection flow. When the application is started, the user must first select and connect to a serial port. The main interface only becomes available after a successful connection has been established. This ensures that the system is ready before any commands can be sent.

In addition, the software supports higher-level functionality such as scheduling medium exchange. The user can create and manage actions through the interface, which are then sent to the firmware for execution. This allows the system to operate automatically over time without continuous user input.

The software stores configuration data locally. This includes user preferences, control settings, and plate configuration. By storing these settings, the software can restore the previous state when the application is restarted, improving usability.

5. Realisation

5.1 Hardware

5.1.1 Left Switch Malfunction

One of the main problems was the left limit switch not working correctly. When the switch was triggered, it caused a short circuit and the Arduino would restart. Because of this, the switch could not be used.

After checking the PCB design, it was found that the trace of the switch signal ran under a 5V trace. In the PCB editor, these connections looked separate (see Figure 5.1), but in practice they were touching (see Figure 5.2). This caused the 5V line to be pulled down when the switch was pressed, which reset the Arduino.



Figure 5.1 L-Switch Electrical Connection viewed from the PCB Editor View

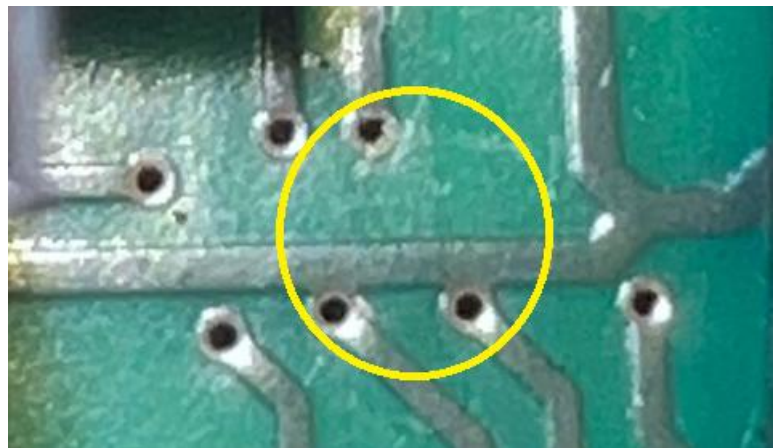


Figure 5.2 L-Switch Electrical Connection viewed in real

To fix this, the faulty PCB trace was cut, and the switch was rewired directly to the correct pins using soldered connections. After this change, the switch worked normally again.

5.1.2 Wiring Improvements

The wiring used jumper wires soldered to pins and covered with tape (see Figure 2.4–2.6). This setup is not reliable for long-term use, especially in an incubator where heat and humidity can damage the tape.

To improve this, the tape was removed and the wiring was redone using crimped connectors with proper headers and plastic covers. This makes the connections stronger and more stable for long-term use.

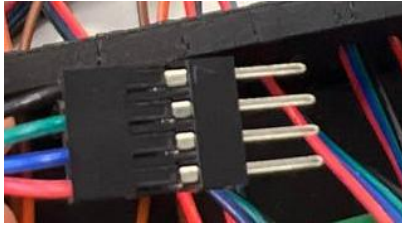


Figure 5.3 Improved wiring with crimped connectors and protective housing

5.1.3 Cable Management

After improving the wiring connections, attention was also given to cable management. The system initially contained loosely arranged wires, which made the setup difficult to handle and increased the risk of tangling or accidental disconnection.

To address this, the cables were grouped and secured using zip ties along their length, as shown in Figure 5.4. This helped to organise the wiring into clear branches and reduced unnecessary movement of cables during operation.

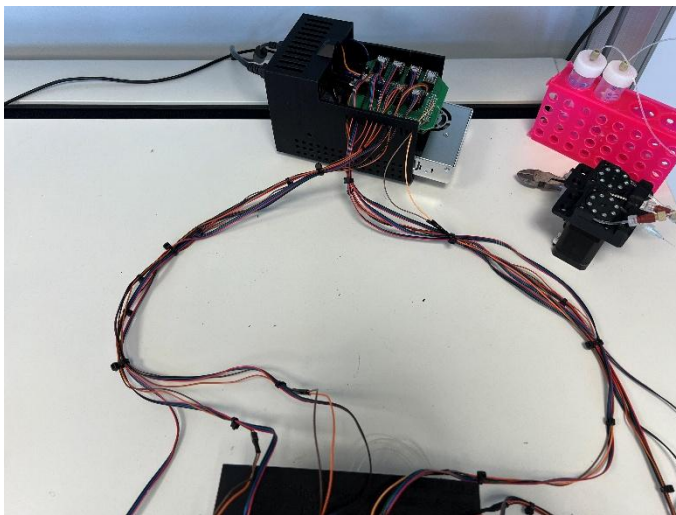


Figure 5.4 Organized wiring with zip ties and grouped cables

The improved cable management makes the system easier to maintain and reduces the chance of mechanical interference or connection issues. It also results in a cleaner and more structured setup.

5.1.4 Motor Wiring Consistency

Another issue was that the motor directions changed after taking the robot apart and assembling it again, which made the system behave unpredictably. After investigating, it was found that the wires inside the motor connectors were not in the same order for all motors (see Figure 5.5). During reassembly, some of these connectors were mixed up, which caused the motors to rotate in the wrong direction.



Figure 5.5 Inconsistent motor wiring order causing incorrect motor direction

To fix this, all motor connectors were opened, and the wire order was made the same for each motor (see Figure 5.6). This ensures that the motors always move in the correct direction, even after future disassembly.

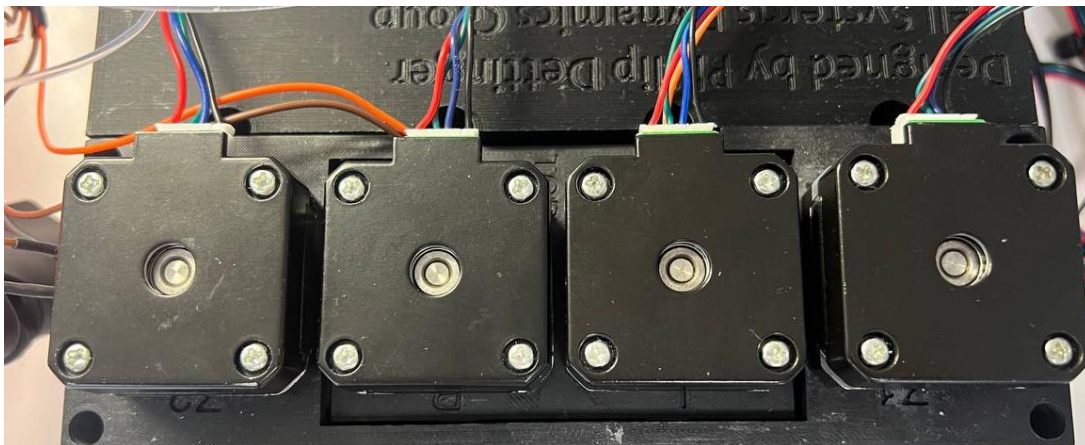


Figure 5.6 Standardised motor wiring order after correction

5.1.5 Arm Column Fixation

Another issue was found in the connection between the stepper motors and the robot arms. The system uses plastic components called arm columns, which are mounted on top of the stepper motor shaft and connect the motor to the arm (see Figure 5.7).



Figure 5.7 Arm column on motor shaft

It was observed that the right motor, which often hit the wall during origin calibration, did not have the fixing screw properly securing the arm column to the motor shaft (see Figure 5.8). Because of this, the metal shaft could rotate inside the arm column without fully transferring the movement to the arm.

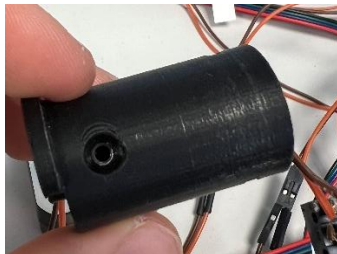


Figure 5.8 Arm column with missing screw

This problem became worse at higher temperatures. The plastic slightly expanded, which made the connection looser. As a result, the motor would rotate, but the arm would either not move or move less than expected.

This also explains why the previous intern observed differences in movement behaviour at higher and lower temperatures [1]. At higher temperatures, the loosened connection affected the accuracy of the movement, while at lower temperatures the connection remained tighter and the system behaved more consistently.

To fix this issue, the fixing screw was properly inserted and tightened to secure the arm column to the motor shaft. After this, the rotation of the motor was fully transferred to the arm, and the problem no longer occurred.

5.1.6 Z-Axis Driver Behaviour

There were also problems with the Z-axis motor drivers. The Z1 and Z2 drivers did not work reliably and had to be temporarily connected to different driver outputs. After improving the wiring, this

problem disappeared. This suggests that the issue was caused by unstable wiring or poor connections, rather than a problem with the motor drivers themselves.

5.1.7 Pumps and Reservoirs Integration

At the start, the pumps and reservoirs were not properly part of the system, which made it impossible to perform automated liquid handling. This was fixed by integrating the pumps and reservoirs into the setup (see Figure 5.9, Figure 5.10). After this, the system was able to perform complete liquid handling operations without manual work.

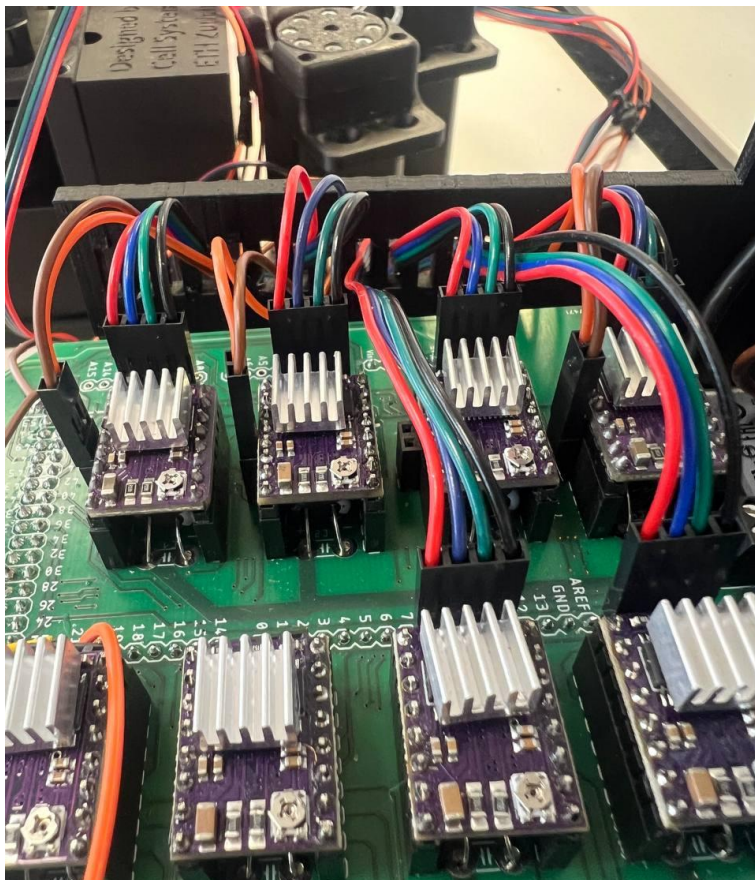


Figure 5.9 Pump Motors connected to Pump Drivers

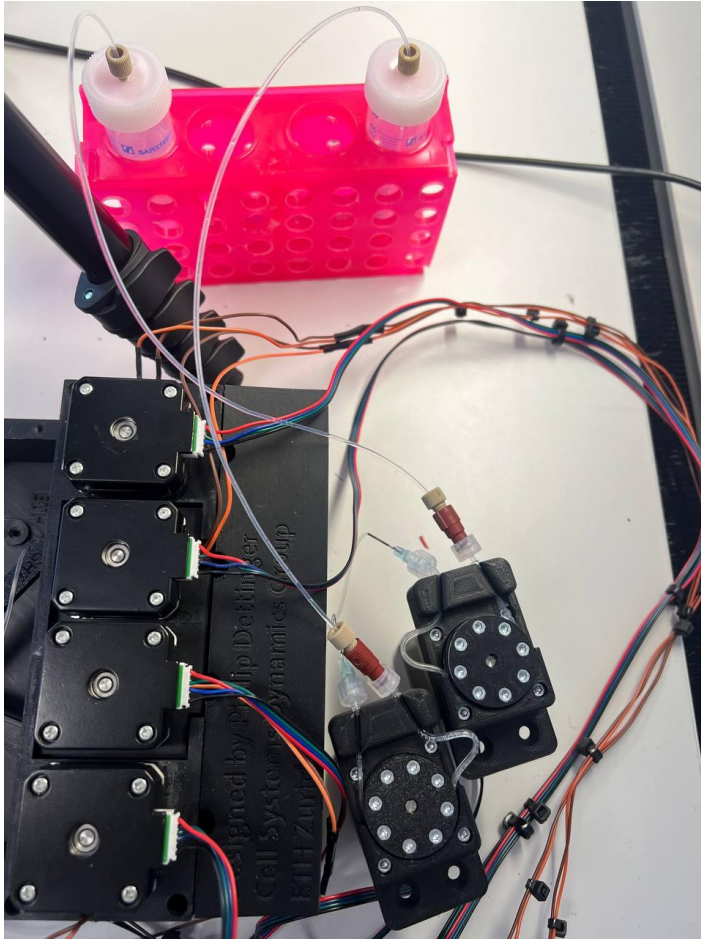


Figure 5.10 Integrated Pumps and Reservoirs

5.1.8 Real-Time Clock Integration

To support scheduling, a RTC module was added to the system (see Figure 5.11). This allows the system to keep track of time even when it is not connected to a computer.

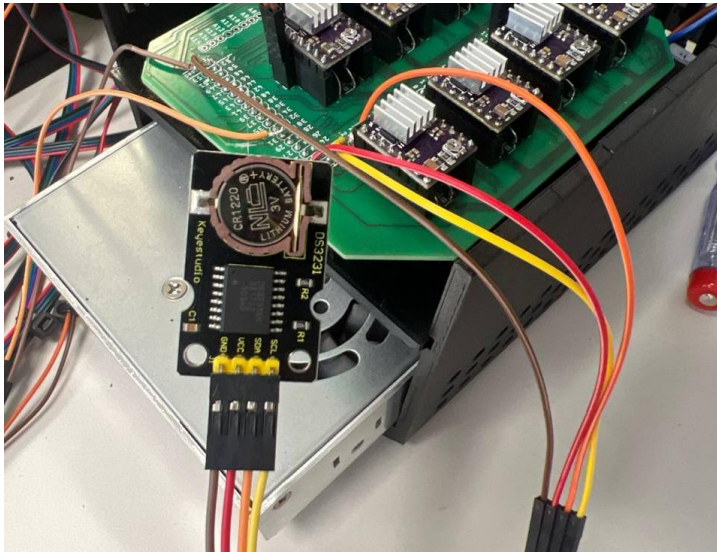


Figure 5.11 Integrated RTC module

The RTC was connected to the Arduino using the I²C interface. With this addition, the system can run scheduled Actions, such as automated medium exchange, at specific times, which makes it more independent and allows it to run for longer periods without user input.

5.2 Firmware

5.2.1 Homing and Origin Calibration

The initial origin calibration procedure relied only on the right limit switch and then continued moving for a short time to ensure alignment. This approach was slow and not reliable, as the final position depended on timing and mechanical contact with a wall.

This was improved by reintroducing the left limit switch and redesigning the initialization logic. The robot first moves to the right until the right limit switch is triggered. It then moves to the left until the left limit switch is triggered. After both limits are known, the system moves a fixed number of steps to position the robot approximately in the center of the working area, as shown in Figure 5.12. This position is then defined as the origin of the system.

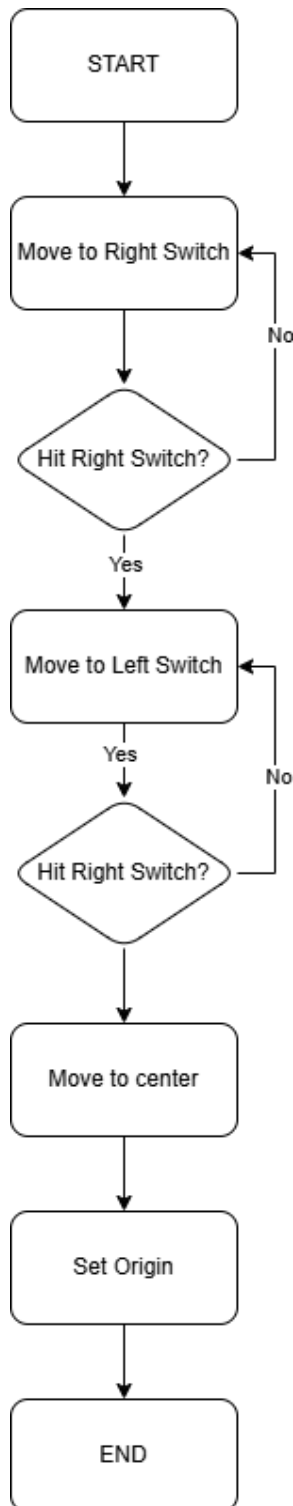


Figure 5.12 Homing and origin calibration procedure based on limit switches

This process effectively calibrates the workspace and establishes a reliable reference position.

After this initial calibration, a separate homing operation can be performed. In this case, the robot simply returns to the previously defined origin without repeating the full limit-switch procedure.

This approach ensures that the system always starts from a well-defined position, without relying on timing or physical pressure against the structure. As a result, initialization became faster, more consistent, and reduced mechanical stress.

5.2.2 Position Persistence (EEPROM)

The robot did not keep track of its position after being turned off. This meant that the system needed to perform homing frequently, even when it was not necessary. To improve this, the positions of all stepper motors (Z1, Z2, L, and R) are now stored in EEPROM whenever a movement is completed. This allows the system to remember its last known position even after a restart.

On startup, the system loads these stored positions and restores the internal state without requiring immediate recalibration. If no valid positions are found in memory, for example after a reset or first use, an origin calibration procedure is performed instead. This ensures that the system always starts from a known position.

Ultimately, this reduces unnecessary origin calibration operations during normal use and makes the system faster and more efficient.

5.2.3 Movement Reliability

Another issue observed in the system was unreliable movement, especially when changing direction. For example, when moving forward and then backward, the software indicated correct motion, but the physical movement did not match. The arm could remain in place before responding correctly, creating a mismatch between the software position and the actual mechanical position.

In addition, the system was observed to behave differently at different temperatures [1]. In some cases, movement appeared more or less accurate depending on the temperature.

Initially, these issues were assumed to be caused by the firmware or motor behaviour. However, further investigation showed that both problems were caused by a mechanical issue in the arm column, as described in Section 5.1.5. Because the motor shaft was not securely connected, small movements were not fully transferred to the arm. At higher temperatures, the plastic expanded, making the connection even looser and increasing the error.

After fixing the arm column by properly securing it to the motor shaft, the movement became consistent and matched the expected software behaviour. The differences observed at different temperatures were no longer present.

5.2.4 Dynamic Well Positioning

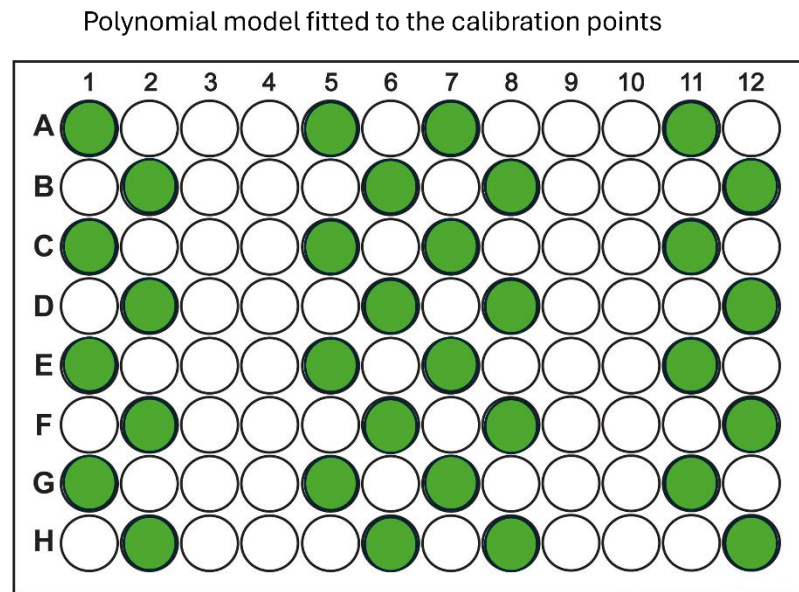
In the previous implementation based on the PHIL system [2], well positions were defined using hardcoded coordinates. These positions are consistent and allow the robot to move to predefined locations. However, this approach is limited, because it only works for a specific plate setup and does not easily support different plate types or configurations.

In the improved version, the hardcoded positions were kept but used as a starting point for calibration. The robot first moves to an approximate well position using these predefined coordinates. The user then manually adjusts the position to the center of the well and records it.

This process is repeated for a selected set of wells distributed across the plate. Once sufficient calibration points are collected, a mapping between well positions and motor coordinates is computed.

These recorded points are then used to fit a polynomial model using a least-squares approach. This model allows the system to estimate motor coordinates for all remaining well positions, instead of relying only on fixed values, as illustrated in Figure 5.13.

-  - Filled circles: measured calibration points
- Unfilled circles: estimated positions



Mapping from grid coordinates (x, y) to motor coordinates

Figure 5.13 Polynomial mapping based on calibration points.

This approach was inspired by previous work [2], where well positions were converted to motor angles using inverse kinematics. However, that method depends on fixed robot dimensions. In this project, a more flexible approach was used.

By using a limited number of calibration points and fitting a polynomial mapping, the system can approximate the full workspace while requiring minimal manual input. This allows the system to adapt to different plate setups and configurations.

As a result, the robot can still use the hardcoded positions for fast calibration, while the fitted polynomial model enables accurate and flexible positioning across a wide range of setups.

5.2.4.1 Pump Control Logic

Pump control was implemented in a similar way to the stepper motor control. Each pump is driven using stepper motors, and movements are defined in steps.

To allow the user to work with meaningful units, a conversion between steps and volume (in microliters, μL) was introduced. The conversion factor was determined through calibration to ensure accurate volume. This makes it possible to specify how much liquid should be aspirated or dispensed, instead of working directly with motor steps.

Two modes of operation are supported. If no well is specified, the selected pump directly aspirates or dispenses the specified volume. This allows simple manual control of the pumps.

If a well is specified, the system performs a coordinated movement. First, the L and R motors move the pipetting arm to the selected well position. Then, the Z motors move the pipette down into the well. Once in position, the pump performs the requested action (aspirate or dispense).

5.2.5 Action-Based Operation

Instead of directly controlling movements and pumps, the system can now execute predefined Actions.

An Action represents a specific task, such as aspirating, dispensing, or exchanging liquid. Each Action contains parameters such as the pump to use, the volume, and timing information.

Actions can be assigned to one or more wells, and each well can have multiple associated actions. This allows operations to be reused across locations while also supporting multiple operations at a single well.

The Action system was designed to use memory efficiently, as the Arduino has limited available memory. For this reason, the total number of actions and their associations to wells are bounded. In the current implementation, a maximum of 64 Actions can be defined, and each well can be associated with up to 16 Actions. These limits ensure that the system remains stable while still providing sufficient flexibility for typical experimental workflows.

In addition, Actions can be scheduled over time. Each Action has an optional start time, optional end time, and optional execution frequency. This enables automated workflows, where operations are performed repeatedly without user interaction.

During operation, the system continuously processes all Active actions. When the conditions for an action are met, it is executed by moving to the corresponding well and performing the required pump operation, as shown in Figure 5.14.

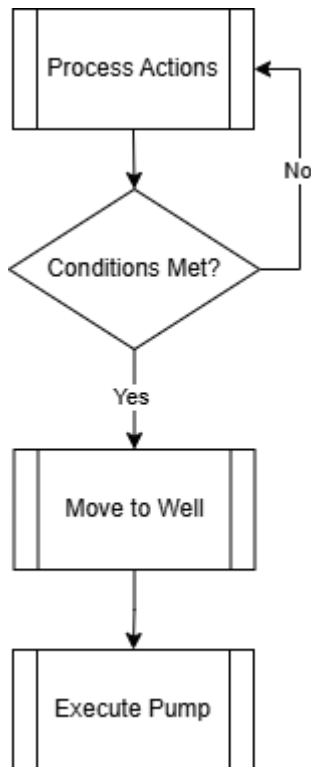


Figure 5.14 Actions Process Diagram

The action data is stored in memory, allowing the system to retain configured workflows between runs.

5.2.6 Communication Interface

Communication between the firmware and the software was improved by using predefined commands with a consistent format. Both the GUI and the firmware follow the same structure when sending and receiving data, ensuring reliable and predictable communication.

Each command represents a specific operation, such as movement, pump control, calibration, or action management. Commands are sent from the GUI to the Arduino, where they are executed by the firmware. Depending on the type of command, the firmware may provide feedback in the form of data updates, such as position changes or selected well, or debug output messages. Not all commands produce explicit structured responses, as some operations only result in internal state changes or simple printed feedback.

The exchanged data includes movement commands, calibration data, well positions, system state information, and command-specific feedback generated by the firmware. By using a fixed command format, both sides interpret the data consistently.

This structured approach makes communication more reliable and easier to extend. New functionality can be added by introducing additional commands without changing the existing communication logic.

The system includes basic error handling mechanisms to ensure safe operation. For example, emergency stop functionality allows immediate interruption of ongoing operations. This structure reduces the risk of incorrect command interpretation.

In addition, validation checks are performed when processing incoming commands to prevent invalid operations or corrupted input from being executed. Commands that do not match the expected format or contain invalid parameters can be safely ignored or rejected.

To further ensure reliable system behaviour, consistent timekeeping between the software and the firmware is required. Therefore, a synchronization mechanism was implemented. If a time mismatch greater than 5 seconds is detected between the PC and the RTC module, the RTC time is automatically updated using the system time from the PC. This ensures reliable execution of time-based actions and prevents drift between the two systems.

5.2.7 Auxiliary Positions

Predefined coordinates were introduced for auxiliary locations such as a wash container and a waste container. These positions are defined using fixed coordinates and can be accessed through dedicated commands, similar to standard movement operations.

This functionality was designed to support extended workflows, such as automated cleaning and liquid disposal during action execution.

However, due to limitations in the current 3D-printed design of the robot, there is no physical space available to integrate these containers into the system. As a result, this functionality is not used in the current implementation and is instead considered for future improvements, as discussed in the recommendations section.

5.2.8 Firmware Refactoring

After the main features were implemented, the firmware was reorganised into separate modules as described in Chapter 4.

This refactoring does not change functionality but improves the structure of the code. By separating responsibilities such as movement, Actions, pump control, communication, and hardware interaction, the system becomes easier to maintain and extend.

The modular structure makes it simpler to add new features in the future, as changes can be made within a single module without affecting the rest of the system. It also improves debugging, since issues can be traced more easily.

5.3 Software

5.3.1 Front-End Redesign

The graphical user interface was redesigned to improve usability and visual consistency. A new white and green theme was introduced, inspired by the ANT logo.

The updated design provides a cleaner layout and clearer separation between different functional areas, making the interface easier to understand and use during operation.

5.3.2 Architecture and MVVM Improvements

The software architecture was improved by refining the MVVM structure. A common ViewModelBase class was introduced, which all view models inherit from.

The ViewModelBase inherits from ObservableObject, providing built-in support for property change notifications through methods such as SetProperty and OnPropertyChanged. This simplifies state management in the user interface and ensures that updates are reflected automatically.

In addition, shared services such as the AppSettingsService and RobotProtocolService are injected into the base class. This allows all view models to access these services without duplicating initialization code.

As a result, repetitive boilerplate code is reduced, and the system becomes more consistent and easier to maintain and extend.

5.3.3 Command Handling and Toolkit Integration

To further reduce boilerplate and improve consistency, the CommunityToolkit.Mvvm library was introduced. This replaces the previously manually implemented RelayCommand logic with a standardized solution.

In addition, the toolkit provides built-in support for dependency injection and observable properties, which simplifies view model implementation and improves code readability.

5.3.4 Services Layer

A dedicated services layer was introduced to separate application logic from the user interface. Several services were implemented, each responsible for a specific part of the system.

The RobotProtocolService handles communication with the firmware and maintains the current state of the robot. The SerialPortService manages the serial connection, including sending and receiving data.

The AppSettingsService manages user configuration by loading and saving application settings. These settings are stored in the AppSettings class, which also provides data required by other parts of the system.

In addition, a MediaService was implemented to support optional video recording during action execution. This service depends on the IRecordContext interface to obtain only the necessary data for recording. By using this interface, the MediaService remains loosely coupled to the rest of the system and does not depend directly on specific implementations such as AppSettings.

This service-based structure allows individual components to be developed and maintained independently, while also improving testability and flexibility of the overall system.

5.3.5 Settings Module

A settings module was implemented to allow users to configure the behaviour of the application. Users can modify parameters such as key bindings and plate type, as shown in Figure 5.15 and Figure 5.16.

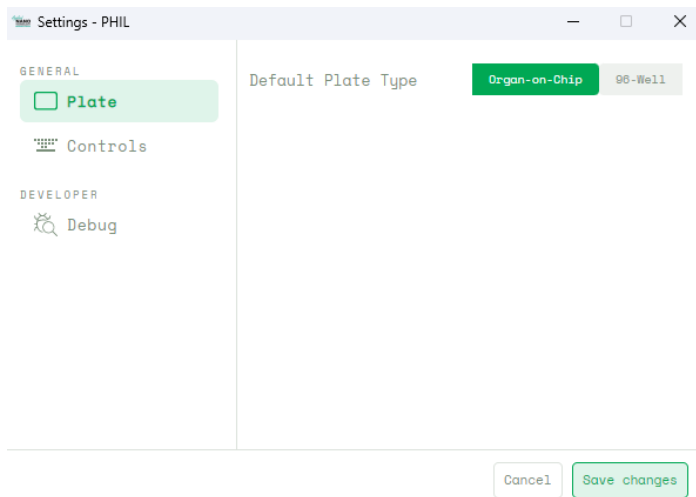


Figure 5.15 Settings Module, PlateView

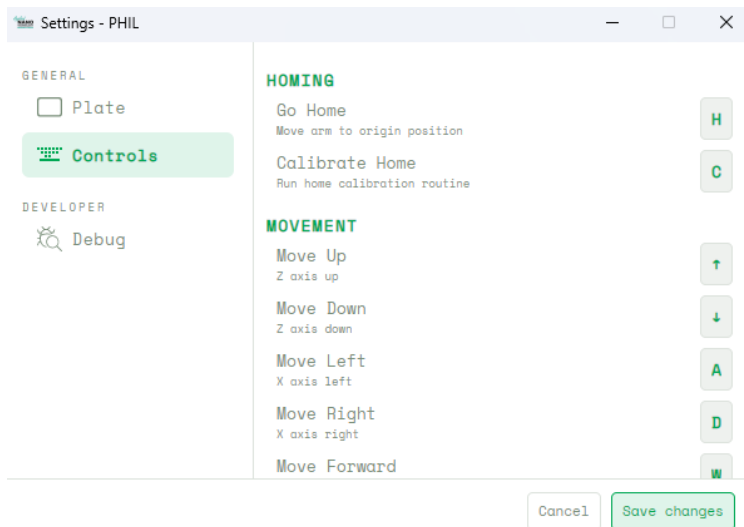


Figure 5.16 Settings Module, ControlsView

The module also provides access to a debug log, which records communication between the GUI and the firmware. This log can be viewed, cleared, or saved to a file for later analysis, as shown in Figure 5.17.

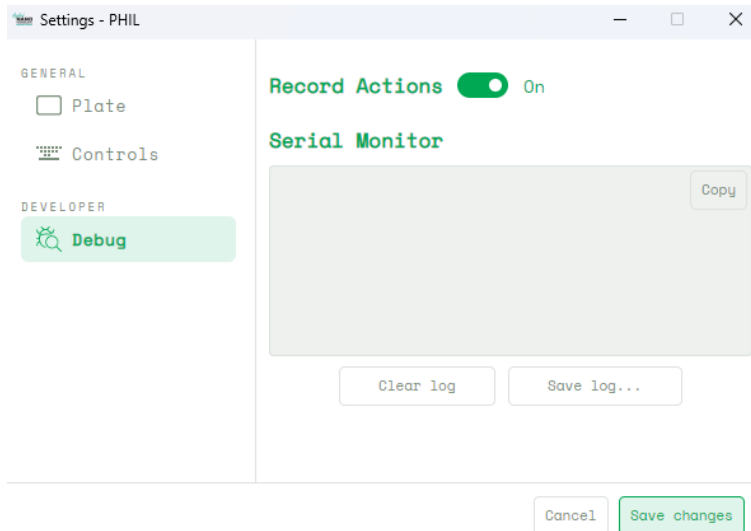


Figure 5.17 Settings Module, DebugView

In addition, users can enable or disable action recording. If enabled and a camera is available, the system records short video clips during action execution.

All settings are stored persistently in a JSON file located in the AppData directory. On startup, the application loads this file. If no configuration file is found, a default configuration is created.

5.3.6 Views and User Interaction

5.3.6.1 WellsView

The WellsView allows the user to interact with the plate after calibration is completed, as shown in Figure 5.18. Access to this view is restricted until calibration is performed, ensuring that movements are accurate.

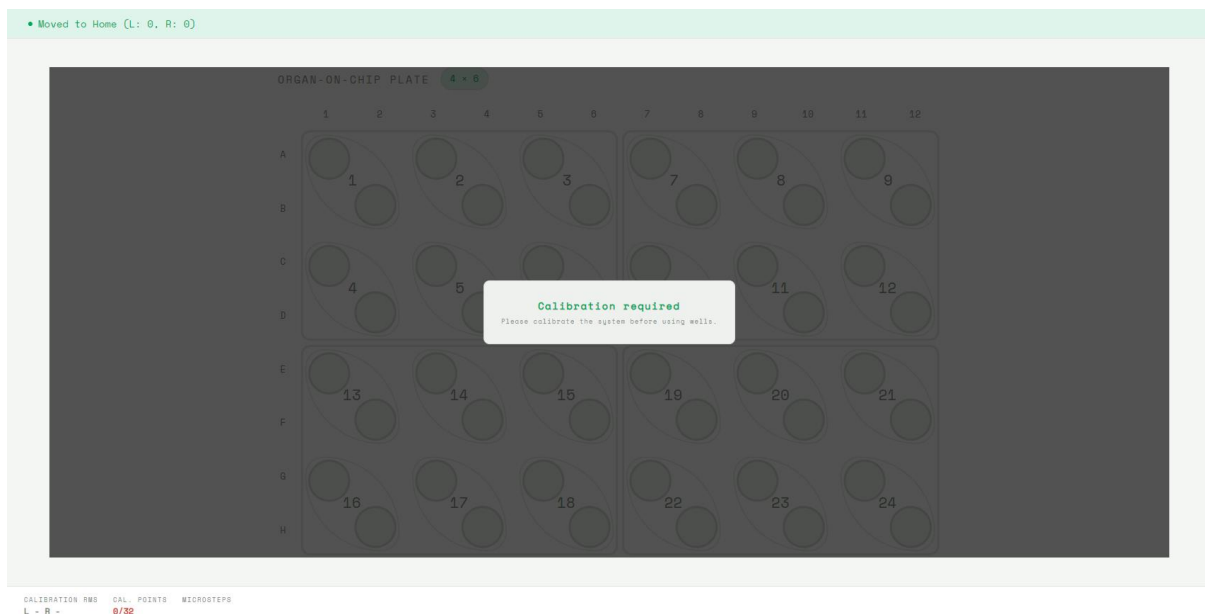


Figure 5.18 WellsView

The plate is implemented using a dynamic ItemsControl, which allows flexible generation of different plate layouts. Users can select wells and perform operations directly through the interface.

5.3.6.2 CalibrationView

The CalibrationView is used to configure the mapping between well positions and robot coordinates. It provides a dynamic plate layout which allows the user to move the robot to the hardcoded well positions using either UI controls or keyboard bindings.

The user can manually position the robot at the center of a well, record calibration points, and compute the mapping. The results can be reviewed directly in the interface, as shown in Figure 5.19.

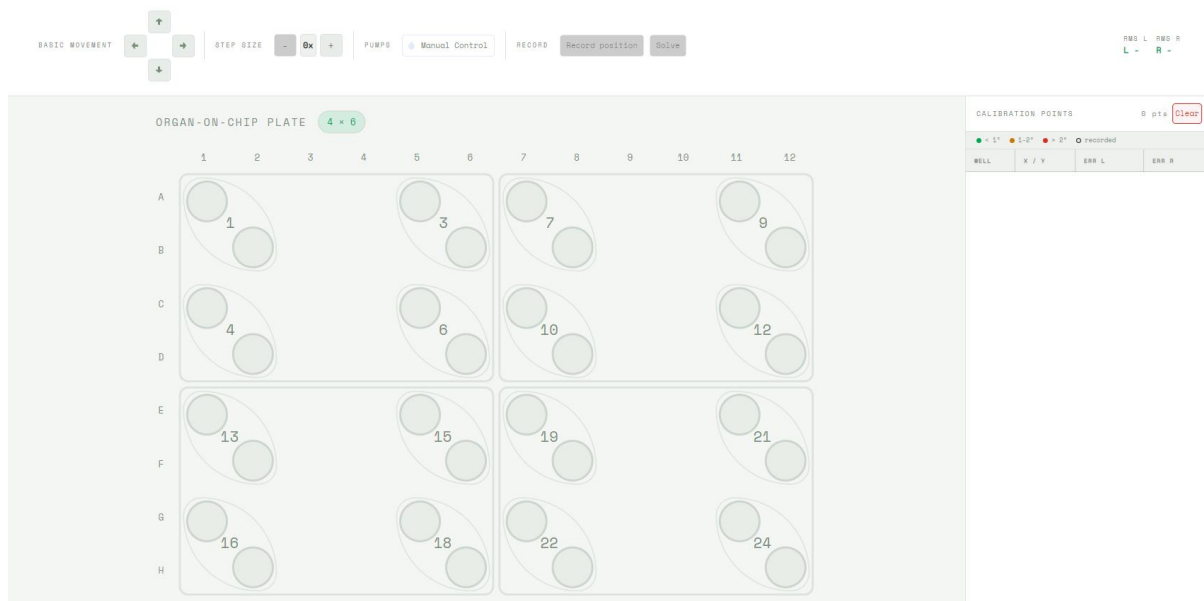


Figure 5.19 CalibrationView

5.3.6.2.1 Pump Control (popup)

A pump control module is available through a popup interface, as shown in Figure 5.20. Users can select a pump, action type (aspirate or dispense), and volume.

In addition, a priming function is provided, which continuously dispenses liquid until manually stopped.

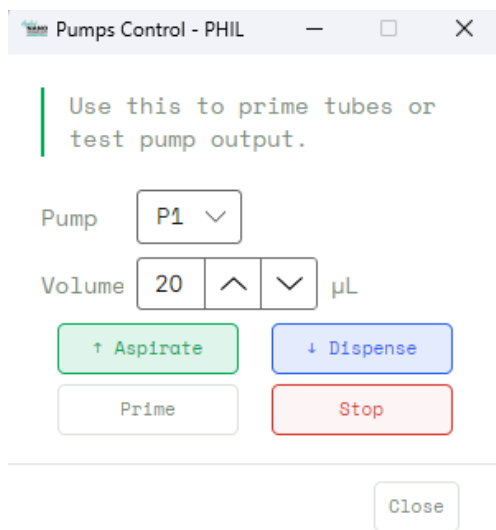


Figure 5.20 Pump Control Window

5.3.6.3 Medium Schedule View

The Medium Schedule view allows users to create and manage Actions, as shown in Figure 5.21. Actions can be assigned to wells using a dynamic plate interface.



Figure 5.21 MediumScheduleView

Each action displays its status, such as active or finished. Users can create or edit actions through a dedicated popup interface, which provides detailed configuration options.

5.3.6.3.1 Action Management (popup)

The Create/Edit Action popup allows the user to configure all parameters of an action, as shown in Figure 5.22 and Figure 5.23. This includes selecting the pump(s), specifying the volume, and choosing the type of operation.

The available options in this window depend on the selected plate type. For the organ-on-chip configuration, both pump-in and pump-out actions can be selected independently (see Figure 5.22). For the 96-well plate configuration, the user selects an action type (aspirate or dispense) together with the pump (see Figure 5.23).

In addition, actions can be configured as repeatable by defining a frequency and time unit. Scheduling options are also provided, where the user can define an optional start time and an optional end time. This enables flexible automation of liquid handling tasks over longer periods.

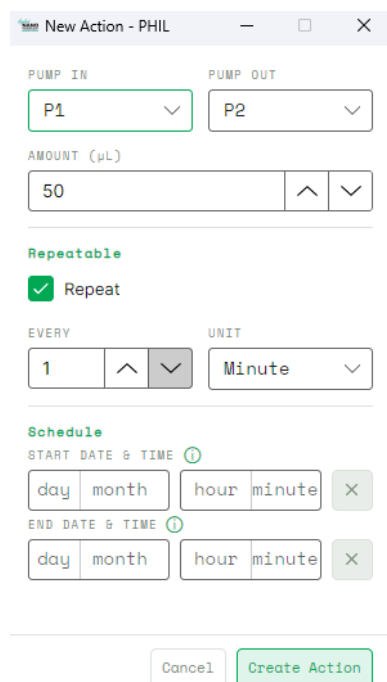


Figure 5.22 ActionWindow for Organ-on-Chip

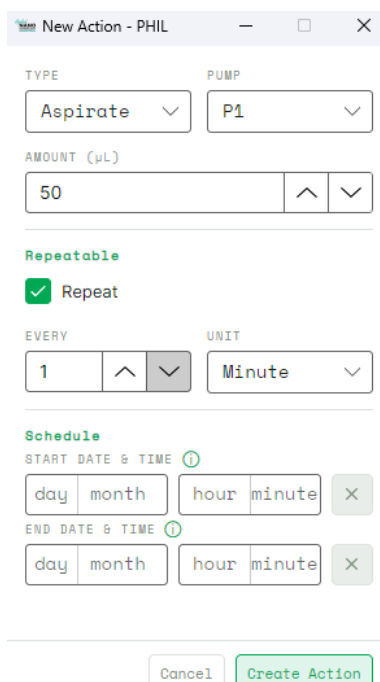


Figure 5.23 ActionWindow for 96-Well

This design provides a clear and structured way to manage complex workflows while keeping the interface intuitive.

5.3.6.4 MainWindow

The MainWindow acts as the central container for all views, as shown in Figure 5.24. Navigation between views is handled through a sidebar, allowing users to switch between different system functionalities easily.

In addition to view navigation, the MainWindow provides access to global controls and system information. These include controls for Z-axis movement, homing, homing calibration, and an emergency stop function, allowing the user to quickly respond to the system during operation.

The sidebar also displays real-time information about the robot state, such as the current well position, X and Y coordinates, and motor angles. This ensures that the user always has insight into the current position of the system.

The settings module is also accessible from the MainWindow, allowing users to configure application behaviour without leaving the main interface.

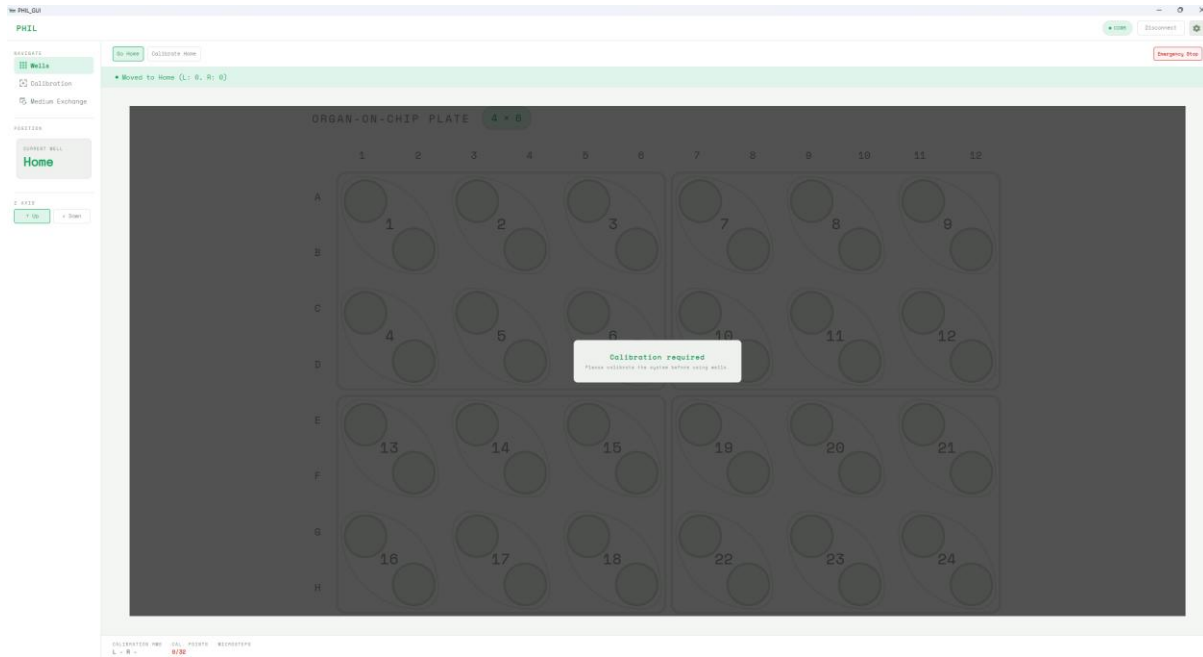


Figure 5.24 MainWindow

5.3.7 Startup and Connection Flow

An initial connection screen was introduced through the PortsWindow, as shown in Figure 5.25. Before accessing the main application, the user must scan for available serial ports and establish a connection to the robot.

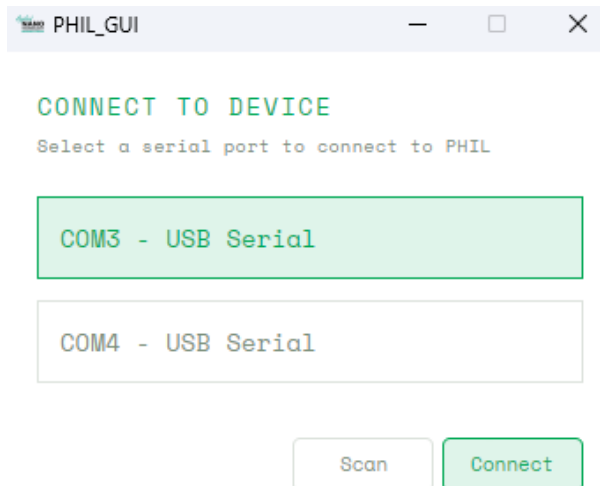


Figure 5.25 PortsWindow

This ensures that the system is properly initialised before any operations are performed.

5.3.8 Documentation and User Support

To support both end users and future developers, additional documentation was created as part of this project.

For end users, a concise handbook was developed to assist ANT employees in operating the PHIL system. This handbook provides step-by-step instructions for common tasks such as connecting to the robot, performing calibration, using pump controls, and configuring automated actions. The goal of this document is to enable new users to quickly become familiar with the system and to provide a reference for resolving common issues.

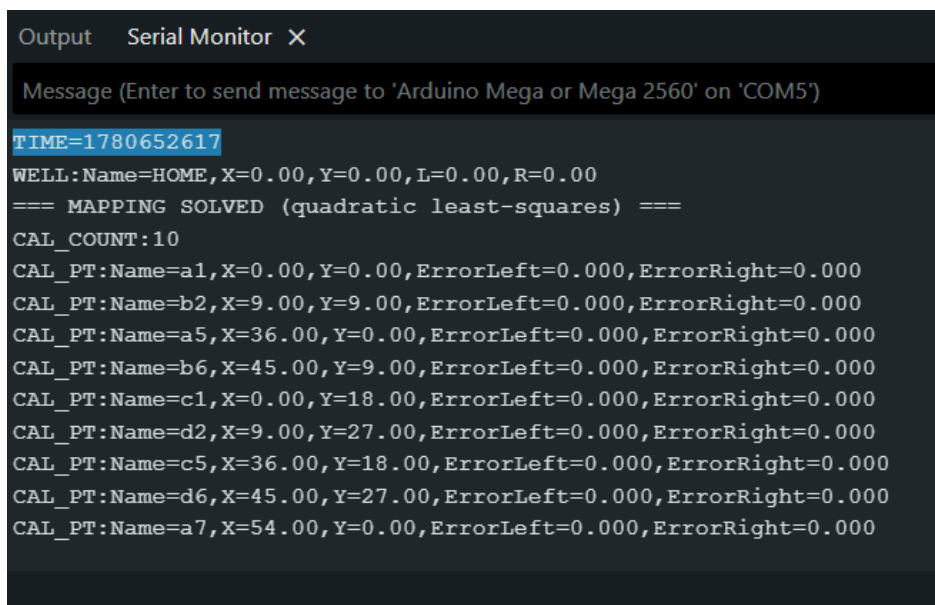
In addition to user documentation, developer-oriented documentation was created as part of this project. This includes adding comments to the firmware and software code, clarifying the purpose of modules and functions, and maintaining a consistent code structure. These improvements make the system easier to understand, maintain, and extend for future development.

6. Testing

6.1 Hardware

6.1.1 RTC Module Test

The real-time clock (RTC) module was tested to verify that it provides consistent and accurate time tracking. The Arduino continuously read the current time from the RTC and printed it through the serial interface, as shown in Figure 6.1.



```

Output  Serial Monitor X
Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM5')
TIME=1780652617
WELL:Name=HOME,X=0.00,Y=0.00,L=0.00,R=0.00
=== MAPPING SOLVED (quadratic least-squares) ===
CAL_COUNT:10
CAL_PT:Name=a1,X=0.00,Y=0.00,ErrorLeft=0.000,ErrorRight=0.000
CAL_PT:Name=b2,X=9.00,Y=9.00,ErrorLeft=0.000,ErrorRight=0.000
CAL_PT:Name=a5,X=36.00,Y=0.00,ErrorLeft=0.000,ErrorRight=0.000
CAL_PT:Name=b6,X=45.00,Y=9.00,ErrorLeft=0.000,ErrorRight=0.000
CAL_PT:Name=c1,X=0.00,Y=18.00,ErrorLeft=0.000,ErrorRight=0.000
CAL_PT:Name=d2,X=9.00,Y=27.00,ErrorLeft=0.000,ErrorRight=0.000
CAL_PT:Name=c5,X=36.00,Y=18.00,ErrorLeft=0.000,ErrorRight=0.000
CAL_PT:Name=d6,X=45.00,Y=27.00,ErrorLeft=0.000,ErrorRight=0.000
CAL_PT:Name=a7,X=54.00,Y=0.00,ErrorLeft=0.000,ErrorRight=0.000
  
```

Figure 6.1 Time Serial Monitor

The reported time values were monitored over an extended period and compared with a reference clock. The results showed that the RTC maintained stable and consistent time without noticeable drift during the observation period.

In addition, synchronization between the PC and RTC module was verified, ensuring correct alignment when discrepancies occur.

6.1.2 Pump Control Test

The pump system was tested to evaluate its basic functionality and responsiveness. Pumps were controlled manually by issuing commands to aspirate and dispense fluid.

During testing, the pumps consistently responded to commands and performed the expected actions. Fluid movement was visually observed to confirm correct operation. No irregular behaviour, such as missed steps or inconsistent motion, was detected.

6.1.3 Volume Calibration (Steps to μL Conversion)

To enable precise liquid handling, a conversion between motor steps and dispensed volume (in microliters, μL) was established through experimental calibration.

A small plastic container (see Figure 6.2) was placed on a precision scale and tared before each measurement (see Figure 6.3). The scale has a resolution of 0.1 mg, corresponding to approximately 0.1 μL assuming a water-like density.



Figure 6.2 Small plastic container



Figure 6.3 Tared scale

The calibration process was performed iteratively. Initial tests were conducted using arbitrary step values, and the resulting dispensed volumes were measured. Based on these results, the step values were gradually adjusted to better match the desired target volumes.

This process was repeated multiple times, generating a large set of intermediate measurements. Only the final calibrated values and validation results are presented in Table 6.4, as they represent the optimized conversion used in the system.

Target Volume (μL)	Measured Values (μL)	Average (μL)	Error (%)
10	7.0, 7.3, 8.6	8.63	-13.7%
20	22.0, 19.8, 18.0	19.9	-0.5%

50	49.5, 50.5, 52.0, 47.7, 51.6, 50.8	50.0	~0%
150	155.6, 156.8, 152.5	155.0	+3.3%
200	200.5, 208.1, 201.0	202.8	+1.4%

Table 6.4 Measured results

The results show that measurement accuracy improves significantly with increasing volume. At low volumes (10 μL), the relative error is high due to the limited resolution of the measurement system and mechanical inaccuracies in the pumping mechanism, particularly at small displacements. For volumes above approximately 20 μL , the system achieves acceptable accuracy, with errors generally below 5%.

Based on these results, a calibration factor was determined to convert motor steps to volume during operation. The final calibration factor was determined to be approximately 1 $\mu\text{L} \approx 9.1$ steps.

6.1.4 Mechanical movement reliability

Mechanical movement reliability was evaluated by repeatedly moving the system between predefined (hardcoded) positions. The robot was commanded to move back and forth between these positions multiple times, while observing whether the position remained consistent.

This test allowed verification of mechanical repeatability and helped identify potential issues such as backlash, slipping, or loss of steps. The system showed consistent positioning behaviour during these movements, indicating reliable mechanical performance under repeated operation.

6.2 Firmware

6.2.1 Homing and Origin Calibration

The homing and origin calibration procedure was tested by executing the full initialization sequence multiple times. Each iteration resulted in the same final origin position, confirming that the firmware correctly processes limit switch inputs and consistently establishes the reference point.

In addition, repeated homing operations (after initialization) correctly returned the system to the stored origin without requiring full recalibration, demonstrating correct state persistence and movement control.

6.2.2 Position Persistence (EEPROM)

Position persistence was verified by writing motor positions to EEPROM and reading them after system restarts. Initial testing confirmed correct storage and retrieval of position values.

To improve data validation, a magic byte (0xCC) was introduced and stored alongside the position data. Upon system startup, the firmware checks this value to determine whether valid data is present in memory. If the expected value is not found, the stored data is considered invalid and a full origin calibration procedure is triggered instead.

This mechanism prevents the system from using uninitialized or corrupted memory, ensuring safe and predictable behaviour.

6.2.3 Dynamic Well Positioning

The dynamic well positioning logic was verified by computing mappings based on recorded calibration points and using them to navigate to intermediate well positions. The computed positions were

consistent and enabled smooth transitions between wells, confirming correct implementation of the interpolation model.

In addition, the recorded calibration points were stored in EEPROM and successfully reloaded after system restart. The use of a validation marker (magic value) ensures that only valid calibration data is used. This prevents the system from operating with uninitialized or corrupted mapping data.

The firmware successfully translates logical well coordinates into motor positions using the computed mapping, demonstrating the correctness of both the position estimation logic and its integration with persistent storage.

6.2.4 Pump Control Logic

The pump control logic was verified by testing the conversion between user-defined volumes and motor steps. The firmware correctly calculated the required number of steps based on the calibrated conversion factor and executed the corresponding pump movements.

Both direct pump control and coordinated operations (movement + pumping) were validated, confirming correct sequencing of Actions within the firmware.

6.2.5 Action-Based Operation

The Action-based system was tested by scheduling Actions with defined start times, durations, and frequencies. The firmware was observed to trigger actions at the correct times and execute them as expected.

In addition, Actions stored in memory were successfully reloaded after system restart, confirming correct integration with EEPROM storage. The use of validation markers (magic values) ensured that only valid action data was processed.

6.2.6 Communication Interface

The communication interface was validated by sending both valid and invalid commands from the software to the firmware. Commands with incorrect parameter counts or malformed structure were correctly ignored, and no unintended actions were executed.

This confirms that the validation logic in the firmware effectively prevents incorrect or unsafe operations caused by invalid input.

6.2.7 Firmware Refactoring

After refactoring, the firmware was tested to verify that all previously implemented functionality remained unchanged. All core operations, including movement, calibration, pump control, and communication, were executed successfully.

6.3 Software

6.3.1 Command Handling and Toolkit Integration

Command handling was tested by triggering various commands through the user interface and verifying that the corresponding actions were executed correctly. The use of the CommunityToolkit.Mvvm library ensured consistent command execution.

6.3.2 Services Layer

The implemented services were tested through their integration within the application. Each service (e.g., RobotProtocolService, SerialPortService, and MediaService) was verified to correctly perform its responsibilities, including communication, state updates, and data access.

The interaction between services was also validated, ensuring that dependencies such as the use of IRecordContext by the MediaService functioned as expected.

6.3.3 Settings Module

The settings module was tested by modifying configuration parameters and verifying that they were correctly saved to and loaded from a JSON file in the AppData directory. After restarting the application, all settings were restored correctly, confirming proper persistence.

In addition, the debug log functionality was tested, including viewing, clearing, and saving logs to a file.

When recording is enabled and a camera is available, the system captures video clips during action execution. This functionality was tested using actions scheduled at different intervals, including short (1 minute) and long-duration (up to 2 days) intervals. All recordings were successfully generated during execution. Example recordings of scheduled actions are provided in the Appendix.

6.3.4 Views and User Interaction

Each view was tested by performing the actions available to the user and verifying that the expected behaviour was achieved. This includes operations such as well selection, calibration, pump control, and Action scheduling.

In addition to functional testing, the software was evaluated by end-users at ANT. Users interacted with the system during normal operation and provided positive feedback regarding usability and functionality. No major issues were reported during this evaluation.

7. Conclusions and Recommendations

7.1 Conclusion

This project focused on improving the PHIL pipetting robot to make it more accurate, reliable, and suitable for automated medium exchange. At the start, the system had several limitations, including unreliable origin calibration, poor positioning accuracy, limited automation, and an incomplete software interface.

During the project, improvements were made in hardware, firmware, and software. On the hardware side, wiring was improved, mechanical issues were fixed, and pumps and reservoirs were integrated into the system. Cable management was also improved, resulting in a cleaner and more stable setup. In addition, a real-time clock was added to support time-based operation.

In the firmware, the homing procedure was redesigned, making it more reliable and repeatable. Position persistence was added using EEPROM, allowing the system to recover its state after a restart. The introduction of dynamic well positioning significantly improved accuracy, while the Action-based system enabled automated operation over time.

On the software side, the interface was redesigned to be clearer and easier to use. A structured architecture using services and MVVM was implemented, improving maintainability and code organisation. Features such as calibration, scheduling, and settings management were added, making the system complete and more user-friendly.

Testing showed that the system performs reliably. Movement is consistent, pump control works as expected, and scheduled Actions are executed correctly over time.

Overall, the main objectives of the project were achieved. The system is now able to perform automated medium exchange with improved accuracy and stability. While some hardware improvements are still recommended, the PHIL system is now suitable for further use and development within the research environment.

7.2 Recommendations

7.2.1 Hardware Improvements

To improve the reliability of the PHIL system, a more robust error detection mechanism should be implemented. One possible approach is to integrate sensors that detect whether the pipette correctly reaches a well or whether there is a mismatch between the intended software position and the actual mechanical position. By incorporating such feedback, the movement logic could be extended to partially rely on sensor input instead of purely calculated positions, increasing positioning accuracy and safety.

These sensors could be integrated into the structure surrounding the well plate. This would require redesigning parts of the 3D-printed body to accommodate the sensors. During this redesign, it is also recommended to select materials that are more resistant to heat, ensuring reliable operation in incubator conditions.

The current well plate holder allows the plate to sit loosely within the structure, which can negatively affect positioning accuracy. A more precise design should be developed to ensure a tighter fit, preventing unwanted movement during operation.

Another issue is related to mechanical deformation. When moving towards the outer rows of the plate, particularly rows 10 to 12, the pipette position becomes lower due to the weight of the arm. This creates a risk of the pipette getting stuck inside a well or colliding with the plate edges. To mitigate this, the base of the 3D structure should be redesigned to slightly elevate the system and compensate for this deflection.

Additionally, the current placement of pumps and reservoirs is not well integrated into the system. These components are currently positioned loosely around the robot. A redesigned 3D structure should include dedicated mounting points for pumps and an organised storage system for reservoirs, improving overall stability and usability. The redesigned body could also include dedicated compartments for a waste container and a cleaning container, positioned for example in the top-left and top-right corners of the structure.

Furthermore, the redesigned body should incorporate dedicated compartments for a waste container and a cleaning (wash) container. This would allow the system to fully support extended workflows, such as automated cleaning and liquid disposal, which are already implemented in the firmware but not currently usable due to hardware limitations. By integrating these components directly into the structure, the overall system would become functionally complete.

7.2.2 Firmware Improvements

With the addition of sensors, the firmware could be extended to support more advanced error handling. By detecting discrepancies between expected and actual positions, the system could respond dynamically and prevent potential collisions or incorrect operations.

Furthermore, extended logging functionality should be implemented. In addition to the existing GUI logs, system-level logs could be stored on an external SD card. This would allow data to be collected over longer periods and provide valuable insight during debugging and performance analysis.

7.2.3 Software Improvements

Currently, well plate configurations in the GUI are hardcoded, which limits flexibility. This functionality can be improved by extending the settings module to allow users to define and modify well plate layouts dynamically. This would make the system adaptable to different experimental setups without requiring code changes.

7.2.4 PCB and Electrical Improvements

The PCB design could also be improved as a lower-priority task. A revised PCB could include support for additional components such as an SD card module and a real-time clock. It could also be extended to support more than four pumps by adding additional driver channels.

Additionally, integrating a DC transformer into the design would allow the system to step down the battery voltage (e.g., 24V) to a stable 5V supply for the VIN input. This would ensure that the system remains powered consistently and safely.

8. Use of Tools

The Dynamic Well Positioning method was inspired by previous research [2]. The implementation was further supported by the use of AI-based tools (such as Claude) to accelerate development and explore suitable mathematical approaches.

Given the limited project duration, this allowed more time to be dedicated to core system features within the domain of firmware and software engineering.

All implemented functionality was integrated, tested, and validated by the author.

9. References

- [1] V. Shvets, "Design and Development of Robotic Pipetting System: Electronics, Firmware, and User Interface," Applied Nanotechnology Research Group, Enschede, Jan. 16, 2025.
- [2] J. Dettinger, J. Dettinger, G. Kull, M. Arekatla, T. Ahmed, Y. Zhang, et al., "Open-source personal pipetting robots with live-cell incubation and microscopy compatibility," Nature Communications, vol. 13, p. 2999, 2022. doi: 10.1038/s41467-022-30643-7.

10. Appendix



Figure 10.1 Action execution recording (1-minute interval)



Figure 10.2 Action execution recording (1-hour interval)



Figure 10.3 Action execution recording (30-minute interval)



Figure 10.4 Action execution recording (2-day interval)